# General Sir John Kotelawala Defence University

# Faculty of Computing



# Essentials of Artificial Intelligence (IT3182, CS22023)

# Group Assignment

# Final Report

| Group Number | Index Number | Name |
|---|---|---|
| 56 | D/COE/25/0021 | EAT Manohara |
| | D/BSE/25/0024 | JNL Asirini |
| | D/BIT/24/0001 | NPGVK Naotunna |
| | D/BIT/24/0058 | PR Serasinghe |

**CONTENT**


01.Abstract ..... 4
2. Introduction ..... 5
3. Problem Background ..... 5
4. Problem Statement ..... 6
5. Objectives ..... 7
5.1 Primary Objective ..... 7
5.2 Secondary Objectives ..... 7
6. Selected AI Techniques ..... 7
6.1 Chromosome Representation ..... 8
6.2 Genetic Algorithm Components ..... 8
6.3 Fitness Function ..... 9
6.3.1 Hard Constraints ..... 9
6.3.2 Soft Constraints ..... 9
6.4 Local Min-Conflict Repair ..... 10
6.5 Interval-Overlap Conflict Detection ..... 11
7. Justification of AI Techniques ..... 11
8. System Design ..... 12
8.1 Genetic Algorithm Workflow ..... 13
8.2 Work Division by System component ..... 14
9. Implementation Details ..... 14
9.1 Data Modelling and Representation ..... 15
9.2 Genetic Algorithm Core ..... 15
9.3 Fitness and Constraint Validation ..... 15
9.4 User Interface, Visualisation and Session State ..... 15
9.5 Export Engine ..... 16
10. Testing and Results ..... 16
10.1 Verified Dataset ..... 16
10.2 Functional Testing ..... 17

10.3 Verified AI Performance ..... 17
10.4 Output Verification ..... 18
11. Limitations and Discussion ..... 21
12. Ethical and Social Considerations ..... 22
12.1 Fairness Across Intakes and Programmes ..... 22
12.2 Transparency and Explainability ..... 22
12.3 Human Oversight ..... 22
12.4 Data Privacy ..... 22
12.5 Workload and Social Impact ..... 23
13. Conclusion ..... 23
14. References ..... 24
15. Individual Contribution ..... 25

# 01.Abstract

University course timetabling is a scheduling problem that has been studied for years. It grows more difficult when multiple student intakes, shared lecturers and shared subjects are scheduled together instead of separately. This report presents the version of a Multi-Intake University Timetable Optimization System. The system was built for Intakes 40 41 42 and 43, across the Computer Science (CS) Software Engineering (SE) Computer Engineering (CE) and Doctor of Business Administration (DBA) programmes.

The system treats timetable construction as a single-fitness-function optimisation problem. It uses a Constraint-Aware Local-Repair Genetic Algorithm. This algorithm combines a Genetic Algorithm with selection, single-point crossover, uniform mutation and elitism.

It also includes a local min-conflict repair step. This step directly fixes hard-constraint violations of depending only on random operators.

Each candidate timetable is represented as a fixed-length chromosome with ten genes. Each gene corresponds to one course. The chromosome is scored using a fitness function. The function is: Fitness = 100,000 × hard violations + soft penalty sum. This ensures that any feasible schedule is always better than any one. Only after feasibility is guaranteed are preferences like earlier start times and balanced daily workload considered.

Conflict detection uses interval-overlap comparison. It checks if start_A < end_B. Start_b < end_A. This method is more accurate than comparing start times. It correctly handles the mix of 1.5-hour 2-hour and 3-hour courses in the dataset.

The system was implemented in Python. It uses Pandas for data handling. It includes a custom Genetic Algorithm and constraint module. It uses Streamlit for the user interface. It exports timetables to PDF and Excel using ReportLab and OpenPyXL.

On a verified dataset with 10 courses, 9 lecturers, 7 rooms and 3 active student groups the system consistently produces timetables with zero violations. These violations include room conflicts, lecturer conflicts, student-group conflicts, capacity conflicts and lecturer-availability conflicts. Typical fitness values are between 12 and 16. Execution times are around 3.5 to 6.2 seconds.

This report covers the problem background the finalised AI techniques, the system design and implementation the testing done the limitations. The ethical considerations, for deploying such a system in a real university setting.

# 2. Introduction

Every semester universities have to deal with the repeated administrative task of creating a timetable that places courses into time slots, rooms and instructors in a way that follows many different rules. When a university has groups of students taking the same or similar degree programs at the same time and when instructors and classes are shared between these groups the scheduling problem can't be handled one group at a time. A change made for Group 42 could cause a problem for Group 41 if the two groups share an instructor or a class. This project focuses on that version of the University Course Timetabling Problem that involves multiple groups.

The system created for this project is a web-based tool that helps with optimization. It takes scheduling data as input in CSV format turns the timetable problem into a problem of optimization with constraints and looks for a timetable that is almost the best possible using a Genetic Algorithm that is made stronger with a step that is done in a certain way. The result is one timetable that's consistent across everything and then it is divided into views for each group checked against strict rules shown in a weekly schedule and made available to download as CSV, PDF and Excel files.

This report is the work, for the project and shows the whole system as it was created, tested and confirmed. It builds on the analysis of the problem that was given at the start and the technical progress that was written about during the review stages.

# 3. Problem Background

Timetabling is one of the combinatorial problems studied in both operations research and artificial intelligence because the number of possible assignments of courses to time slots and rooms grows combinatorially with the number of courses, rooms and time slots making exhaustive search impractical even for modestly sized datasets. In a university context the problem is further complicated by additional factors that are present in this projects target environment.

First the project covers four intakes (40, 41 42 and 43) and three to four degree programmes (CS, SE, CE and DBA) that must ultimately be scheduled as one connected system than four independent ones. Several lecturers teach across than one intake and at least one course in the active dataset (COE22032, Computer Interfacing and Microprocessors) is a shared lecture attended jointly by multiple intake-programme groups. A lecturer-level or room-level conflict can therefore only be detected correctly if all intakes are considered together.

Second courses, in the dataset do not all have the length. The verified dataset includes courses that're 1.5 hours, 2 hours and 3 hours long. A scheduling system that only checks start times and ignores the duration of each course might fail to detect real overlaps. This happens because two courses could start at times but still run at the same time if their total lengths are different. So

checking the start times is not enough. We need to compare the full time intervals to be sure there are no conflicts. This observation directly motivated the interval-overlap conflict rule described in Section 5.

Third producing a timetable manually for this scale of problem is slow and error-prone. A human scheduler has to mentally track room capacities, lecturer availability across intakes and student-group workload and small changes (for example moving one class) can silently introduce a new conflict elsewhere. This is the motivation, for automating the process with a search-based AI technique.

# 4. Problem Statement

We have a set of courses each with a duration, a teacher assigned and one or more student groups connected to it. We also have a list of rooms each with a fixed number of seats. There is a group of teachers who cannot teach two classes at the time. We are working with 30-minute time blocks from Monday to Friday starting at 08:00 and ending at 17:00. Additionally we have a list of student groups, each defined by the year they started and the program they are in along, with the number of students enrolled. The goal is to assign each course to one room and one time block so that all constraints are met.

- No room is given to two courses that are scheduled at the same time (room clash).
- No teacher is given to two courses that are scheduled at the time even if the courses are for different groups (lecturer clash).
- No student group is given to two courses that are scheduled at the time (student-group clash).
- Every course is placed in a room that can hold all the students attending (capacity violation).
- Every course is scheduled during the time when the teacher's available to teach (lecturer availability violation).

At the time we want to make sure that the schedule follows some softer preferences: putting classes earlier in the day rather than later not having classes end after 16:30 and not having more than two classes for the same student group on the same day. The strict rules above must always be followed in any schedule. The softer rules are only used to compare and choose the possible schedule. This setup turns the timetable problem into a goal of reducing the number of issues, which is perfect, for a Genetic Algorithm.

# 5. Objectives

## 5.1 Primary Objective

The goal is to create and use an AI-based system that automatically makes one consistent multi-intake university timetable. The timetable must obey all scheduling rules reduce soft scheduling penalties and show the result via an interactive web interface with export options.

## 5.2 Secondary Objectives

• Prevent clashes involving rooms, lecturers and student groups across the combined schedule rather than only within one intake.

•. Detect conflicts for courses that last different amounts of time using interval-based overlap detection.

• Show shared lectures that are attended by than one intake-programme group as one timetable entry not as duplicates.

• Improve timetable quality by following preferences: start earlier avoid late finishes and balance daily class load for each student group.

• Resolve any remaining conflicts that appear during the search by using a deterministic local min-conflict repair step instead of relying only on random mutation.

• Provide filtered timetable views for each intake that come from one global optimisation run.

• Provide a to-use Streamlit web interface for entering data generating the timetable visualising the timetable and validating the timetable.

• Support dynamic lunch-break placement that is aware of the generation process of a single fixed lunch slot.

• Allow the generated timetable to be exported as CSV, PDF and a multi-sheet Excel workbook, for use by the institution.

# 6. Selected AI Techniques

Finalized AI approach is a Constraint-Aware Local-Repair Genetic Algorithm. This technique was chosen due to the fact that the University Course Timetabling Problem has many interacting scheduling constraints, and a very large search space. In fact, an exhaustive search would be unrealistic due to the number of combinations as the courses, rooms and available timeslots increase in possible number. The implemented GA finds good timetable assignments by maintaining a population of candidate timetables and continually improves the timetables

through evolutionary operations. The current implementation, however, is not a purely generic Genetic Algorithm, as it combines hard-constraint awareness and hard- constraint deterministic repair.

The overall technique can be considered as three connected components:

1. **Genetic search**, which explores the timetable solution space.
2. **Constraint-aware fitness evaluation**, which strongly penalizes invalid solutions.
3. **Local min-conflict repair**, which directly improves conflicting assignments.

## 6.1 Chromosome Representation

A chromosome has 10 genes, one for each course. Each gene holds the scheduling information needed to show a course assignment. Since each course is in the chromosome once a chromosome shows a full timetable candidate, for the current dataset.

## 6.2 Genetic Algorithm Components

| **GA component** | **Implemented value** |
|---|---|
| Population size | 50 |
| Number of generations | 50 |
| Selection | Elitist rank selection |
| Parent proportions | Top 50% |
| Crossover | Single-point |
| Crossover rate | 1.0 |
| Mutation | Uniform point mutation |
| Mutation rate | 0.1 |
| Elitism | Best individual preserved |
| Local repair | Deterministic mini – conflict repair |

*Table 6.1 — Finalised Genetic Algorithm configuration.*

## 6.3 Fitness Function

### 6.3.1 Hard Constraints

| Hard Constrains | Description |
|---|---|
| Room clash | Two courses cannot occupy the same room during overlapping intervals |
| Lecturer clash | A lecturer cannot teach overlapping courses |
| Student-group clash | A student group cannot attend overlapping courses |
| Room capacity | Assigned room must have sufficient capacity |
| Lecturer availability | Assigned teaching interval must respect lecturer availability |

*Table 6.2— Hard constraints enforced by the fitness function.*

Violation of each of these hard constraints is punishable by 100,000. This weighting assures that feasibility is given a higher priority than the optimization objective. So a schedule that carries a lower soft penalty and has one or more hard penalties is much worse than a feasible schedule.

Variable course duration is a primary reason for the use of the interval-overlap rule. If course A starts before course B ends and course B starts before course A ends, the classes overlap:

start_A < end_B AND start_B < end_A

This rule is used wherever time conflicts need to be identified.

### 6.3.2 Soft Constraints

After the system checks if something is possible it works on making the schedule by adding some soft rules. These rules are about making sure that classes start early not ending late and not having too many classes in one part of the day.

The penalty for starting time increases. It starts at 0 for 08:00 to 08:30. Goes up, to 30 if the class starts at 16:00 or later. This helps to choose times without making the later times completely wrong.

Additional +10 penalty for courses completed after 16:30. Concentration (daily basis): +10 for three classes in one day and +20 for four or more classes in one day.

All these values are incorporated into the fitness function:

Fitness = 100,000 × total of hard violations + sum of soft penalties

In practical terms, this objective is hierarchical: towards feasibility, and then towards quality of the timetable.

| Start Period | Penalty |
|---|---|
| 08:00 – 08:30 | 0 |
| 09:00 – 09:30 | 1 |
| 10:00 – 10:30 | 2 |
| 11:00 – 11:30 | 3 |
| 12:00 – 12:30 | 4 |
| 13:00 – 13:30 | 5 |
| 14:00 – 14:30 | 6 |
| 15:00 – 15:30 | 15 |
| 16:00 and later | 30 |

*Table 6.3 — Soft start-time penalty schedule.*

## 6.4 Local Min-Conflict Repair

The part where we fix things locally makes the genetic algorithm better by focusing on conflicts instead of just depending on the usual changes that happen in evolution. If a schedule has problems the fixing part looks for ways to assign things and picks the ones that make the problems as small, as possible.

The method works in an fixed way instead of being completely random. This matters because scheduling usually has a lot of rules and random changes might keep causing the same issues over and over. Local repair therefore provides a direct mechanism for moving a candidate toward a more feasible region of the search space. The combination of global search and local repair is particularly relevant to the present problem. The GA supplies exploration of different timetable patterns, while local repair provides focused improvement when a candidate contains undesirable assignments.

### 6.5 Interval-Overlap Conflict Detection

Because courses in the dataset range from 1.5 to 3 hours, all overlap checks in the fitness function and the repair module use the full time interval of each course rather than only its start time:

$$\text{start_A} < \text{end_B} \quad \text{AND} \quad \text{start_B} < \text{end_A}$$

Two bookings are considered to overlap exactly when this condition holds. This single rule is reused for room clashes, lecturer clashes and student-group clashes, giving the system one consistent and technically sound definition of a scheduling conflict regardless of course length.

## 7. Justification of AI Techniques

Genetic Algorithms are a known and widely studied method for solving educational timetabling problems. They work well because they do not need the search space to be smooth or easy to calculate and they can handle data like the ten-gene chromosome used in this project. With four intakes, several programs, shared subjects and shared lecturers the number of timetables becomes extremely large. A brute-force approach would take too long and manual scheduling is too slow and error-prone. Genetic Algorithms help by exploring timetables at the same time and gradually improving them over generations.

A rule-based fitness function was chosen of a simple weighted scoring system. This is because university timetabling has constraints that're not equal in importance. For example a room being double-booked is completely unacceptable no matter what.. A class starting a few minutes late is just not ideal. The fitness function was designed to reflect this: 100,000 times the number of rule violations plus the sum of soft penalties. This ensures the algorithm first finds timetables and only then focuses on making them better.

A pure Genetic Algorithm without repair can struggle when many of the candidate timetables are invalid. Crossover and mutation alone may keep producing schedules that break rules. Adding a local-repair step helps fix the specific conflicts in each candidate. This speeds up the process by

making it easier to reach a timetable with low penalties as seen in the fitness improvement graph from Section 10.

In the end all three methods used. Search-based optimization, a constraint-driven fitness model and local repair. Are techniques in artificial intelligence. These methods are not machine learning models that learn from data. Instead they rely on known rules. This is the choice for a problem defined by strict rules rather than patterns in data. Since there is no labeled dataset for timetabling using these techniques avoids the need, for data that simply does not exist.

# 8. System Design

The system is divided into four parts: a user interface layer built using Streamlit, a Python application backend that holds the optimisation part, a timetable-management part that comes between the basic GA results and the views shown to users and an output and export part. Figure 8.1 shows the design, including which team member is, in charge of each part of the backend.

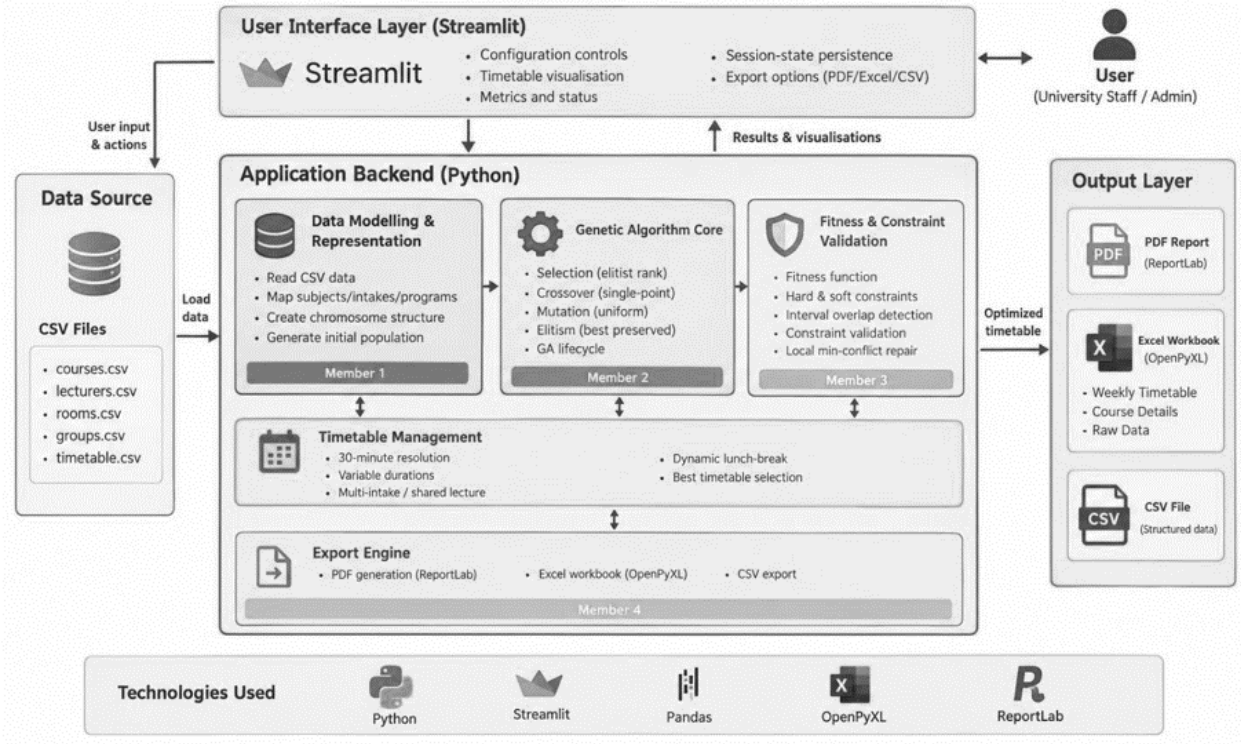
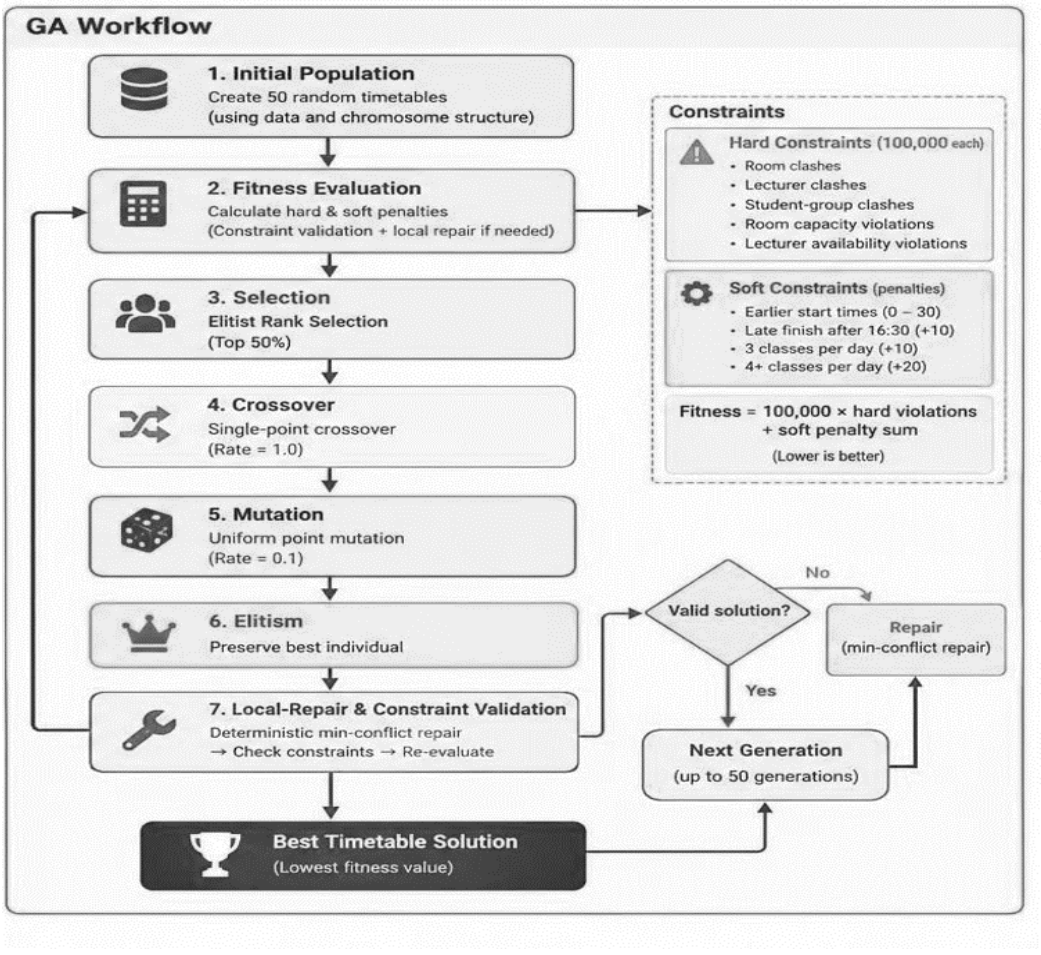



*Figure 8.1 — Finalised system architecture, showing data flow from CSV input through the Streamlit UI, the Genetic Algorithm core, fitness and constraint validation, timetable management, the export engine, and out to the user as PDF, Excel and CSV output.*

Data enters the system as CSV files that contain courses, lecturers, rooms, student groups and time slots. These CSV files are loaded, checked for errors and can be edited through the Streamlit interface. When a user types or clicks something the action goes to the application backend. In that backend the Data Modelling layer creates the chromosome structure and the initial population. The Genetic Algorithm core then carries out selection, crossover, mutation and elitism. After that the Fitness and Constraint Validation module rates every candidate and if a candidate does not meet the rules it triggers repair. Next the Timetable Management layer manages 30-minute slots adjusts courses that last different lengths, handles courses that share a lecture or have intakes and places lunch breaks flexibly. Finally the results and visualisations come back, to the Streamlit UI. If needed are sent to the Export Engine to create PDF, Excel and CSV files.

## 8.1 Genetic Algorithm Workflow

Figure 8.2 shows the generational loop in detail, including where constraint checking and repair sit relative to selection, crossover, mutation and elitism.



### 8.2 Work Division by System component

Responsibilities were split so that every member was, in charge of a clearly defined part of the architecture. At the time everyone took part in putting the entire system together and testing it.

| Member | System Component | Key Files |
|---|---|---|
| P.R. Serasinghe | CSV data modelling, subject/intake/program mapping, timetable and chromosome representation, initial population generation. | data/*.csv, timetable.py, generator.py |
| N.P.G.V. Naotunna | Genetic Algorithm core: selection, crossover, mutation, elitism and the main GA lifecycle. | genetic_algorithm.py, main.py |
| T.M. Edirisinghe | Fitness calculation, hard and soft constraints, interval-overlap detection, constraint validation, local min-conflict repair. | fitness.py, constraints.py |
| J.N.L. Asirini | Streamlit UI, timetable visualisation, dynamic lunch-break handling, session-state persistence, PDF/Excel/CSV export. | app.py, ui_optimizer.py, export_engine.py |

# 9. Implementation Details

The system is written in Python 3. It uses Pandas to work with tables of data. A special Genetic Algorithm was made on purpose and also a module to check constraints. No other optimization software was used so the team could have control over how the constraints work. Streamlit is used to create a web page where people can interact with the system. ReportLab is used to make PDF files and OpenPyXL is used to create Excel files. The code is divided into a small parts. Each part is managed by one person, on the team. This made it easy to put everything together.

## 9.1 Data Modelling and Representation

Courses, lecturers, rooms, student groups and time slots are read from CSV files into Pandas DataFrames. The timetable structure and chromosome design in timetable.py create an object made up of TimetableEntry records one for each gene. The initial population generator in generator.py creates a set of possible timetables. These candidate timetables are then run through a repair step, before the main generational loop starts.

## 9.2 Genetic Algorithm Core

The GeneticAlgorithm class in genetic_algorithm.py pre-computes lookup dictionaries for course details room capacities, valid lecturer IDs and time-slot information including day start and end times. This pre-computation helps avoid repeated queries to the Pandas DataFrames during fitness evaluation and repair. It's a performance optimization because evaluation happens per candidate in each generation. The class provides methods, like create_initial_population() evaluate_population() select_parents() using rank selection (top 50%) crossover() using single-point crossover and mutation and elitism logic. These components together form one generation of the algorithm lifecycle, which is managed by main.py.

## 9.3 Fitness and Constraint Validation

The fitness and constraints module carries out the interval-overlap rule that is explained in Section 5.5. It uses this rule consistently to find conflicts between rooms, lecturers and student groups. It checks if the room capacity is enough for the size of the student group involved. It also checks if the lecturer is available. Additionally it applies the start-time late-finish and daily-concentration penalties that are described in Section 5.3.2. The deterministic local min-conflict repair function is also part of this module. This function is used when creating the population and also, after each generations fitness evaluation.

## 9.4 User Interface, Visualisation and Session State

The Streamlit front end (app.py) shows tables for Intakes, Degree Programs, Intake & Program Groups, Subjects, Subject–Intake–Program mapping, Rooms, Lecturers and Time Slots. The Streamlit front end also runs validation checks. Displays a shared-lecture preview. When the user clicks Generate Optimized Timetable, the code in ui_optimizer.py runs generate_optimized_timetable(). The function generate_optimized_timetable() builds a GeneticAlgorithm runs the generation loop and keeps the timetable ever found instead of just the last generation's best. The function generate_optimized_timetable() also saves the fitness history for the improvement chart. Adds a constraint-validation report to the output. Because Streamlit re-runs the script each time a user does something the timetable, the fitness history and the validation report are kept in st.session_state. This keeps them available when a user changes tabs or downloads a file of losing or regenerating them. A Reset button clears this state on purpose. A

main visual feature is lunch-break placement. Of locking in one fixed lunch slot, find_lunch_slots_for_week() looks for a single slot that is free on all five days. Find_lunch_slots_for_week() first checks 12:00–12:30, other half-hour options. If no single slot works for every day the system chooses a lunch slot for each day. The weekly grid is then created with build_weekly_grid(). Shown as an HTML table, by render_weekly_grid_html(). The lunch row is highlighted so it stands out from the teaching rows.

### 9.5 Export Engine

export_engine.py takes the timetable data and creates three different output formats. First it produces a CSV file with the raw schedule data. Next it builds an Excel workbook using OpenPyXL. This workbook includes three sheets: one for the weekly timetable one for course details and one for the raw data. The Excel file also features merged header cells and some styling. Finally it generates a landscape A4 PDF using ReportLab. This PDF includes the university header, a grid, for the weekly timetable and a course legend.

During development a problem came up. OpenPyXL throws an error when you try to write a value into a cell that is part of a merged range. To solve this a helper function called write_cell_block() was created. This function writes each value into the left cell of a block first and then the surrounding cells are merged after that. This approach prevented a MergedCell crash that had been occurring.

## 10. Testing and Results

### 10.1 Verified Dataset

Testing was done on the dataset that was confirmed to be correct. The dataset includes 10 courses, 9 lecturers, 7 rooms and 3 student groups that are currently active. The student groups are 42_CS with 28 students, 42_SE with 30 students and 42_CE with 24 students. There are 90 time slots that're each half an hour long. These time slots are from Monday to Friday between 08:00 and 17:00. The courses in the dataset last from 1.5 hours like Research Writing Skills, up to 3 hours like Group Project in Software Development and Engineering Drawing. These durations were picked to test the logic for overlapping time intervals. One course, called Computer Interfacing and Microprocessors which has the code COE22032 is a lecture that is shared among groups. Lecturer L003 is teaching two courses, Software Architecture and Software Project Management. This part of the dataset is used to test if the system can detect when a lecturer is scheduled for than one course at the same time. The system was built to handle four groups, which are called Intake 40 Intake 41 Intake 42 and Intake 43. However the current dataset has most of its information filled in for Intake 42. Some entries are also included for the

groups when possible. This is considered a working model that's ready for multiple groups but not a complete dataset, for all four groups.

## 10.2 Functional Testing

Each exported module was tested separately before being put. The data-loading and chromosome-representation module was checked using the CSV files. This ensured that variable course durations and intake/programme mappings were parsed correctly. The GA core was run through main.py as a command-line entry point. This verified that the generational loop, selection, crossover, mutation and elitism all worked as expected. The fitness and constraint module was tested with sample assignments that were deliberately conflicting. This confirmed that violations related to room, lecturer, student-group, capacity and availability were detected and fixed properly. The Streamlit interface, visualisation and export engine were tested from start to finish. A timetable was generated the weekly grid and legend were. Each of the outputs. CSV, PDF and Excel. Were downloaded and opened to make sure they were correct.

During end-, to-end testing the full application was also run without a user interface using Streamlit's automated testing system. This confirmed that clicking the Generate Optimized Timetable button completed without any runtime errors and produced a populated results table. It also confirmed that main.py ran to completion when executed from the command line. Both of these checks passed on the codebase.

## 10.3 Verified AI Performance

In verification runs on the active data set the system always produced feasible schedules, with the following attributes:

| **Metric** | **Result** |
|---|---|
| Room clashes | 0 |
| Lecturer clashes | 0 |
| Student-group clashes | 0 |
| Capacity violations | 0 |
| Lecturer availability violations | 0 |

| | |
|---|---|
| Typical final fitness | 12 – 16 |
| Earliest start time | 08:00 |
| Latest start time | 14:00 |
| Latest finish time | 16:00 |
| Average start time | ≈ 09:15 |
| Typical execution time | ≈ 3.5 – 6.2 seconds |

*Table 10.1 — Verified AI performance on the active dataset.*

Figure 10.1 shows a representative fitness-improvement curve: the population's best fitness falls in clear steps as better-repaired, lower-penalty timetables are found and preserved by elitism, converging to a stable low value well before the generation budget is exhausted.




*Figure 10.1 — Fitness improvement across generations, and the Export Timetable Central Hub.*

## 10.4 Output Verification

The following screenshots from the running application document the verified end-to-end workflow of successful generation and constraint validation (Figure 10.2), the complete unified timetable with course, lecturer and room detail (Figure 10.3), the weekly visual timetable with the dynamically placed lunch break (Figure 9.4), and the course legend with CSV/PDF/Excel export controls (Figure 10.5).

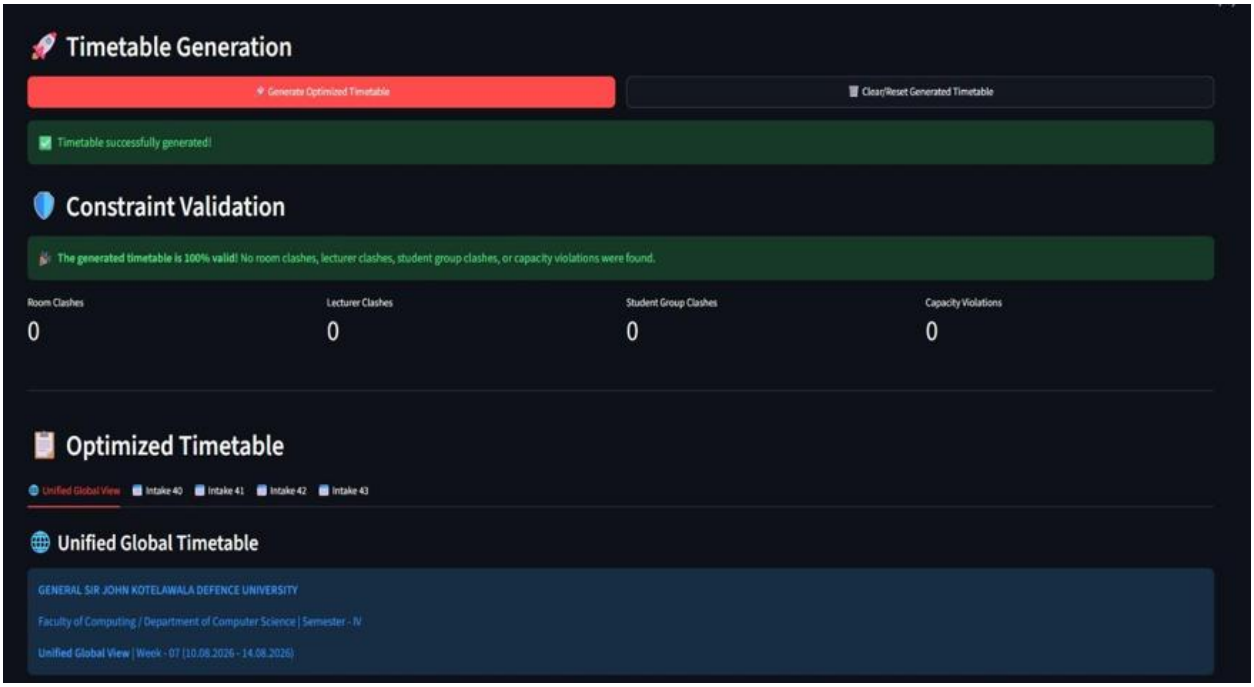
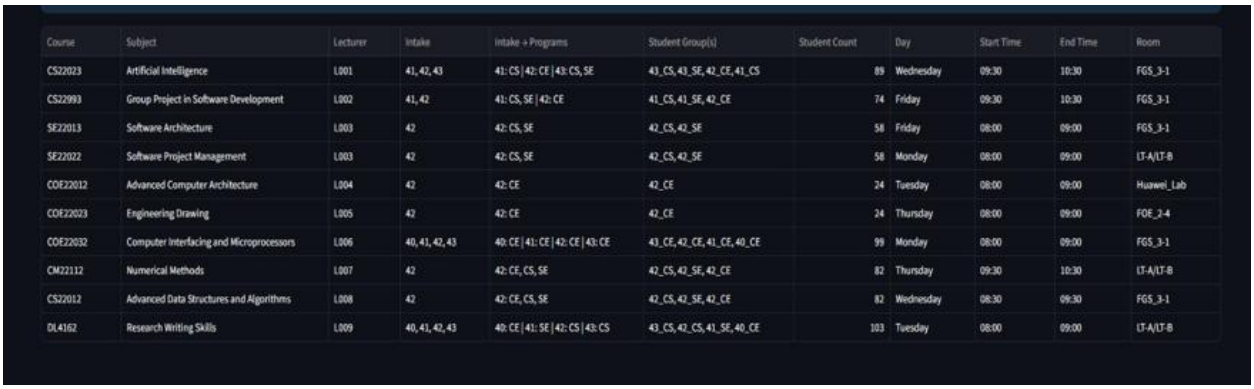



*Figure 10.2 — Timetable generation and constraint validation: zero violations across all four checked hard constraints.*



| Course | Subject | Lecturer | Intake | Intake → Programs | Student Group(s) | Student Count | Day | Start Time | End Time | Room |
|---|---|---|---|---|---|---|---|---|---|---|
| CS22023 | Artificial Intelligence | L001 | 41, 42, 43 | 41: CS \| 42: CE \| 43: CS, SE | 43_CS, 43_SE, 42_CE, 41_CS | 89 | Wednesday | 09:30 | 10:30 | FGS_3-1 |
| CS22993 | Group Project in Software Development | L002 | 41, 42 | 41: CS, SE \| 42: CE | 41_CS, 41_SE, 42_CE | 74 | Friday | 09:30 | 10:30 | FGS_3-1 |
| SE22013 | Software Architecture | L003 | 42 | 42: CS, SE | 42_CS, 42_SE | 58 | Friday | 08:00 | 09:00 | FGS_3-1 |
| SE22022 | Software Project Management | L003 | 42 | 42: CS, SE | 42_CS, 42_SE | 58 | Monday | 08:00 | 09:00 | LT-A/LT-B |
| COE22012 | Advanced Computer Architecture | L004 | 42 | 42: CE | 42_CE | 24 | Tuesday | 08:00 | 09:00 | Huawei_Lab |
| COE22023 | Engineering Drawing | L005 | 42 | 42: CE | 42_CE | 24 | Thursday | 08:00 | 09:00 | FOE_2-4 |
| COE22032 | Computer Interfacing and Microprocessors | L006 | 40, 41, 42, 43 | 40: CE \| 41: CE \| 42: CE \| 43: CE | 43_CE, 42_CE, 41_CE, 40_CE | 99 | Monday | 08:00 | 09:00 | FGS_3-1 |
| CM22112 | Numerical Methods | L007 | 42 | 42: CE, CS, SE | 42_CS, 42_SE, 42_CE | 82 | Thursday | 09:30 | 10:30 | LT-A/LT-B |
| CS22012 | Advanced Data Structures and Algorithms | L008 | 42 | 42: CE, CS, SE | 42_CS, 42_SE, 42_CE | 82 | Wednesday | 08:30 | 09:30 | FGS_3-1 |
| DL4162 | Research Writing Skills | L009 | 40, 41, 42, 43 | 40: CE \| 41: SE \| 42: CS \| 43: CS | 43_CS, 42_CS, 41_SE, 40_CE | 103 | Tuesday | 08:00 | 09:00 | LT-A/LT-B |

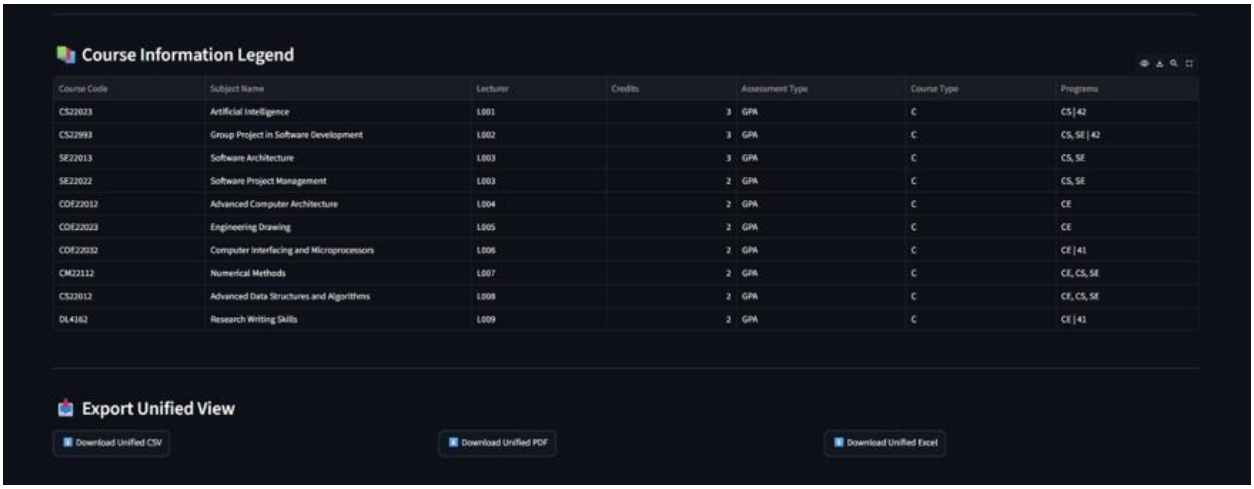
*Figure 10.3 — Unified global timetable with course, lecturer, intake→programme mapping, student counts, scheduling and room detail.*



Unified Weekly Visual Timetable

| Time Slot | Monday | Tuesday | Wednesday | Thursday | Friday |
|---|---|---|---|---|---|
| 08:00 - 08:30 | Computer Interfacing and Microprocessors<br>FGS_3-1<br>L006<br>40: CE \| 41: CE \| 42: CE \| 43: CE | Research Writing Skills<br>LT-A/LT-B<br>L009<br>40: CE \| 41: SE \| 42: CS \| 43: CS | - | Engineering Drawing<br>FOE_2-4<br>L005<br>42: CE | Software Architecture<br>FGS_3-1<br>L003<br>42: CS, SE |
| 08:30 - 09:00 | | | Advanced Data Structures and Algorithms<br>FGS_3-1<br>L008<br>42: CE, CS, SE | | |
| 09:00 - 09:30 | - | - | | - | - |
| 09:30 - 10:00 | - | - | Artificial Intelligence<br>FGS_3-1<br>L001<br>41: CS \| 42: CE \| 43: CS, SE | Numerical Methods<br>LT-A/LT-B<br>L007<br>42: CE, CS, SE | Group Project in Software Development<br>FGS_3-1<br>L002<br>41: CS, SE \| 42: CE |
| 10:00 - 10:30 | - | - | | | |
| 10:30 - 11:00 | - | - | - | - | - |
| 11:00 - 11:30 | - | - | - | - | - |
| 11:30 - 12:00 | - | - | - | - | - |
| 12:00 - 12:30 | LUNCH BREAK | | | | |
| 12:30 - 13:00 | - | - | - | - | - |
| 13:00 - 13:30 | - | - | - | - | - |
| 13:30 - 14:00 | - | - | - | - | - |
| 14:00 - 14:30 | - | - | - | - | - |
| 14:30 - 15:00 | - | - | - | - | - |
| 15:00 - 15:30 | - | - | - | - | - |

*Figure 10.4 — Unified weekly visual timetable, Monday to Friday, with the dynamically allocated lunch break shown on the 12:00–12:30 row.*



Course Information Legend

| Course Code | Subject Name | Lecturer | Credits | Assessment Type | Course Type | Programs |
|---|---|---|---|---|---|---|
| CS22023 | Artificial Intelligence | L001 | 3 | GPA | C | CS \| 42 |
| CS22993 | Group Project in Software Development | L002 | 3 | GPA | C | CS, SE \| 42 |
| SE22013 | Software Architecture | L003 | 3 | GPA | C | CS, SE |
| SE22022 | Software Project Management | L003 | 2 | GPA | C | CS, SE |
| COE22012 | Advanced Computer Architecture | L004 | 2 | GPA | C | CE |
| COE22023 | Engineering Drawing | L005 | 2 | GPA | C | CE |
| COE22032 | Computer Interfacing and Microprocessors | L006 | 2 | GPA | C | CE \| 41 |
| CM22112 | Numerical Methods | L007 | 2 | GPA | C | CE, CS, SE |
| CS22012 | Advanced Data Structures and Algorithms | L008 | 2 | GPA | C | CE, CS, SE |
| DL4162 | Research Writing Skills | L009 | 2 | GPA | C | CE \| 41 |

Export Unified View

Download Unified CSV | Download Unified PDF | Download Unified Excel

*Figure 10.5 — Course information legend and the Export Unified View controls (CSV / PDF / Excel).*

# 11. Limitations and Discussion

Several technical issues were fixed during development and they are listed here for completeness. An Excel MergedCell crash happened when trying to write values into merged cells during export. This was fixed by using the write_cell_block() helper function, which is explained in Section 8.5. The optimiser at first put many classes in the early morning because early start times had a very low penalty. This problem was solved by adjusting the start-time penalty schedule shown in Table 5.3. Clash detection that only looked at start times did not work properly for courses with different lengths. This was corrected by switching to interval-based overlap detection, across the system. Timetable data generated by the system was lost every time the Streamlit app restarted. This was fixed by using st.session_state to keep track of the data. A fixed lunch period sometimes overlapped with scheduled classes. This issue was resolved using the lunch-slot search described in Section 8.4.

There are still limitations in the current verified version of the system. The active CSV dataset is still mainly filled for Intake 42 with a little coverage for Intakes 40 41 and 43 through shared lectures. The architecture can handle all four intakes. A fully filled four-intake dataset has not yet been tested from start to finish. The Genetic Algorithm has a fixed population size and a fixed number of generations in the application code of being user-configurable. This makes it hard to experiment with how fast the algorithm converges or how long it takes without changing the code. Likewise the Streamlit interface hard-codes tabs for Intakes 40 to 43 of creating them on the fly from the intakes that are present, in the loaded data. That means the system would need to be changed before it could work with a set of intakes.

On the testing side verification was done using one fixed dataset and a few runs. It did not include a strong set of repeated trials with different random seeds. The consistency of fitness values and execution time across runs has not been fully checked. Also how the system behaves when the number of courses, rooms and intakes increases has not been formally studied.

A small issue with the code structure was found. The export module has a fallback path that imports helper functions from the UI module. This creates a one-way dependency from the export logic back to the UI layer. To fix this the shared functions for building lunch schedules and grids should be moved into their independent module. This would remove the dependency. Improve the code structure.

In the future it would also be good to add automated regression tests, for the fitness functions and the constraint-checking functions. This would help catch problems and ensure that changes do not break existing behavior.

# 12. Ethical and Social Considerations

Automating a decision that used to need human administrative judgment brings up ethical and social issues that were considered while building this system. These issues need to be looked at more before the system is actually put to use.

## 12.1 Fairness Across Intakes and Programmes

Since all intakes and programmes use the fitness function and the same pool of rooms and lecturers there is a risk that the optimiser could treat some groups unfairly. For example one intake might get more convenient time slots just because of the order in which courses are processed. The soft-penalty design described in Section 5.3.2 applies the start-time and daily-load penalties to every student group. This is meant to stop this kind of treatment.. This should be checked carefully once the system is used with a full dataset that has multiple intakes. Fairness that works on a test dataset might not work when the system is used with all the data.

## 12.2 Transparency and Explainability

A Genetic Algorithm does not give a reason for why a particular course ends up in a particular timeslot; it only returns the best solution it found. To help with this the system shows the hard-constraint validation outcome (room, lecturer, student-group and capacity checks each shown as either satisfied or violated) in the interface instead of only a single opaque score. This allows an administrator to see exactly which constraints were checked and confirm that all of them passed, even though the exact optimisation trade-offs behind each slot assignment are not individually explained.

## 12.3 Human Oversight

The system is created to support decisions not to make them on its own. All scheduling data is. Can be edited by a human administrator before the timetable is generated and the created timetable is shown for review along with its complete validation report before it is exported and used. This keeps a human in the loop for approval, which is suitable because real-world constraints such, as a lecturers personal circumstances a late room-booking change or an accessibility requirement might exist outside the data that the Genetic Algorithm receives.

## 12.4 Data Privacy

The datasets used in this project refer to lecturers and student groups using anonymised identifiers such as L001, 42_CS of full personal names. These datasets do not include any student records. They only include group sizes. If this system were expanded to use lecturer names or individual student data in a production environment that data would need to be

managed according to the universitys data protection rules. Access to that data would be limited to authorised staff.

### 12.5 Workload and Social Impact

The goal of this system is to reduce the time and manual work needed for an administrative task. It does not aim to replace academic decisions. Scheduling policies, such as which soft preferences are most important are still. Approved by university staff. The computational cost of the method is low. A population-based search of 50 individuals over 50 generations, on a dataset of this size takes a few seconds on regular hardware. Therefore the system does not add energy or infrastructure costs compared to manual scheduling.

## 13. Conclusion

This project has produced a Multi-Intake University Timetable Optimization System. The Multi-Intake University Timetable Optimization System views the scheduling of four intakes and many degree programmes as one linked optimisation problem, not as problems for each intake.

The main AI contribution is a Constraint-Aware Local-Repair Genetic Algorithm. The Constraint-Aware Local-Repair Genetic Algorithm uses a fitness function that values feasibility zero violations in room, lecturer, student-group, capacity and availability before it values soft scheduling quality. The Constraint-Aware Local-Repair Genetic Algorithm also uses local repair that fixes conflicts directly instead of depending on chance. Additionally the Constraint-Aware Local-Repair Genetic Algorithm uses interval-based overlap detection that correctly handles courses of lengths.

On the verified dataset the Multi-Intake University Timetable Optimization System consistently creates fully feasible timetables. These timetables have soft-penalty totals and are generated in only a few seconds. The results appear for the user in a Streamlit interface. The Streamlit interface allows data editing gives constraint validation feedback shows a weekly visual timetable with dynamic lunch-break placement and lets the user export in CSV, PDF and Excel formats.

The implementation and this report together show how the original proposal has evolved. They document the progress-review changes. Lead to a verified end-to-end optimisation system. That optimisation system has a representation, a constraint model, a search strategy, a repair process, a user interface and an export facility.

The limitations noted in Section 10 the need to fully populate the four-intake dataset and to make GA parameters user-configurable are the clearest paths, for extending this work beyond the current submission.

# 14. References


[1] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: University of Michigan Press, 1975.

[2] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.

[3] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.

[4] N. Pillay, "A review of hyper-heuristics for educational timetabling," *Annals of Operations Research*, vol. 239, no. 1, pp. 3–38, 2016.

[5] S. Abdullah and H. Turabieh, "On the use of multi neighbourhood structures within a Tabu-based memetic approach to university timetabling problems," *Information Sciences*, vol. 191, pp. 146–168, 2012.

[6] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[7] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io.

[8] The pandas development team, "pandas documentation." [Online]. Available: https://pandas.pydata.org/docs/.

[9] OpenPyXL, "OpenPyXL documentation." [Online]. Available: https://openpyxl.readthedocs.io.

[10] ReportLab, "ReportLab documentation." [Online]. Available: https://docs.reportlab.com.

# 15. Individual Contribution

**ESSENTIALS OF ARTIFICIAL INTELLIGENCE**

**INDIVIDUAL CONTRIBUTION SHEET**

| | | | |
|---|---|---|---|
| **Student Name** | P.R.Serasinghe | **Registration No.** | D/BIT/24/0058 |
| **Degree / Intake** | IT - Intake 41 | **Group No.** | 56 |
| **Project Title** | Intelligent Multi-Intake University Timetable Optimization System Using Genetic Algorithms | | |

## Technical Contributions :

| Task / Component | My Contribution | Tools / AI Technique Used | Evidence / Output |
|---|---|---|---|
| CSV data modelling | Designed the CSV input files for courses, lecturers, rooms, student groups and timeslots. | Python, Pandas (DataFrame loading and validation), CSV | data/courses.csv, lecturers.csv, rooms.csv, student_groups.csv, timeslots.csv |
| Subject / Intake / Program mapping | Mapped each subject to its intake and program groups (e.g. 41_CS, 41_SE) and cleaned the mapping data. | Python, Pandas, dictionary lookup (mappings_by_course), data normalisation | format_intake_program_mapping() in src/timetable.py; mapping logic in src/generator.py |
| Timetable and | Designed the TimetableEntry (gene) and Timetable | | TimetableEntry and Timetable classes in src/timetable.py |

| | | | |
|---|---|---|---|
| chromosome representation | (chromosome) classes with copy support. | Python OOP, Genetic Algorithm encoding (gene / chromosome design) | |
| Initial population generation | Wrote generate_random_timetable() to create random timetables that seed the GA population. | Python, random module, randomised initialisation (GA population seeding), constraint-aware time filtering | generate_random_timetable() in src/generator.py; used by src/genetic_algorithm.py |
| Integration and testing | Tested the data loading and mapping against the real curriculum data. | Python, unit and validation scripts | tests/test_flexible_mapping.py, tests/test_official_curriculum.py, tests/verify_realdata.py |

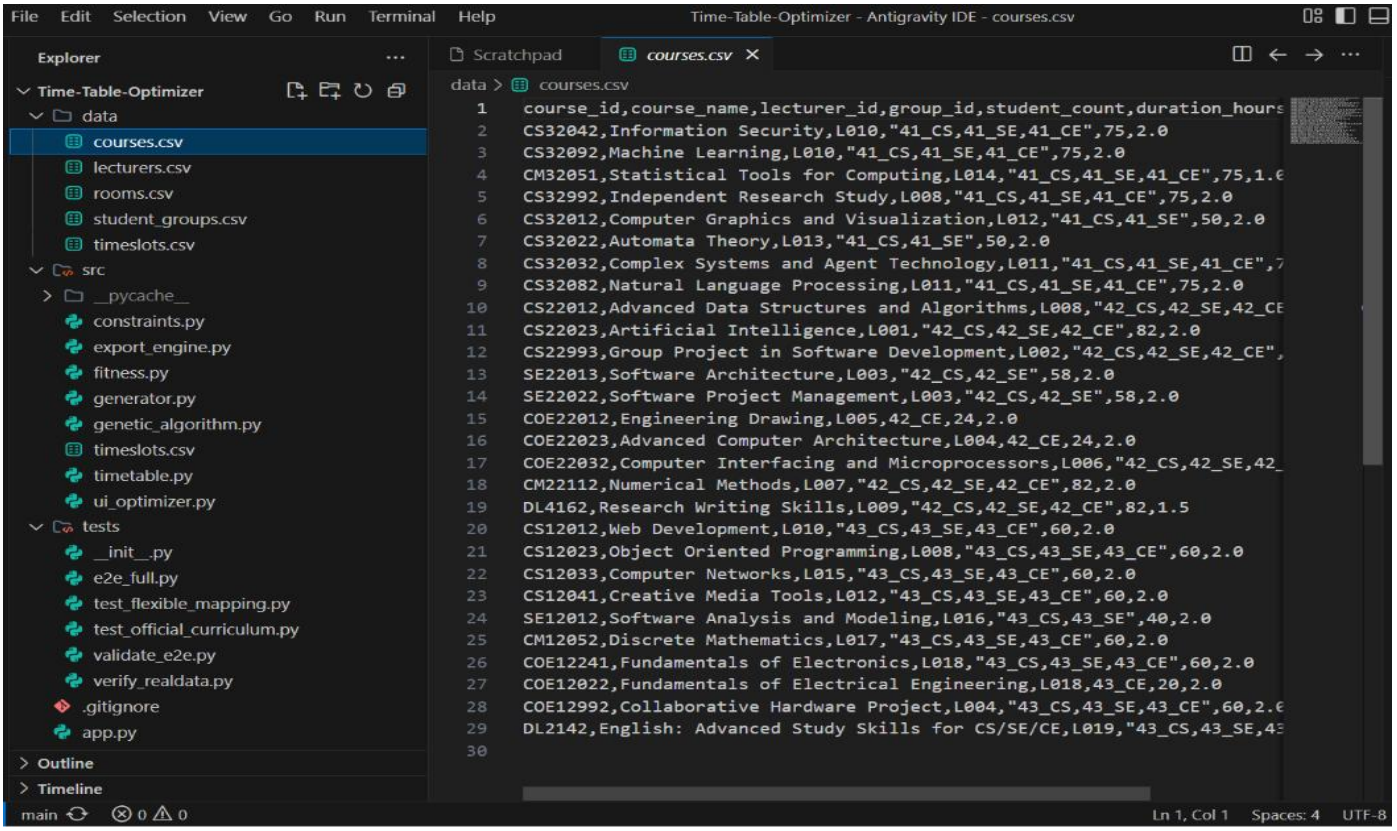
**Declaration**

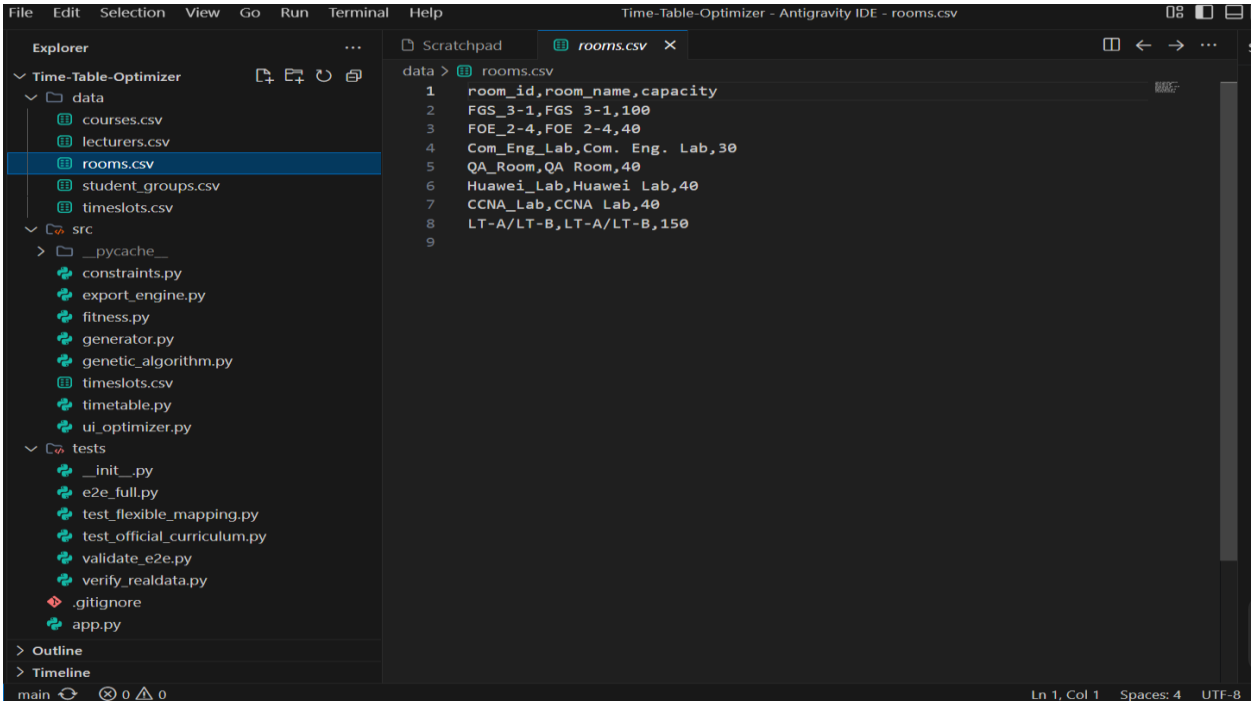
I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

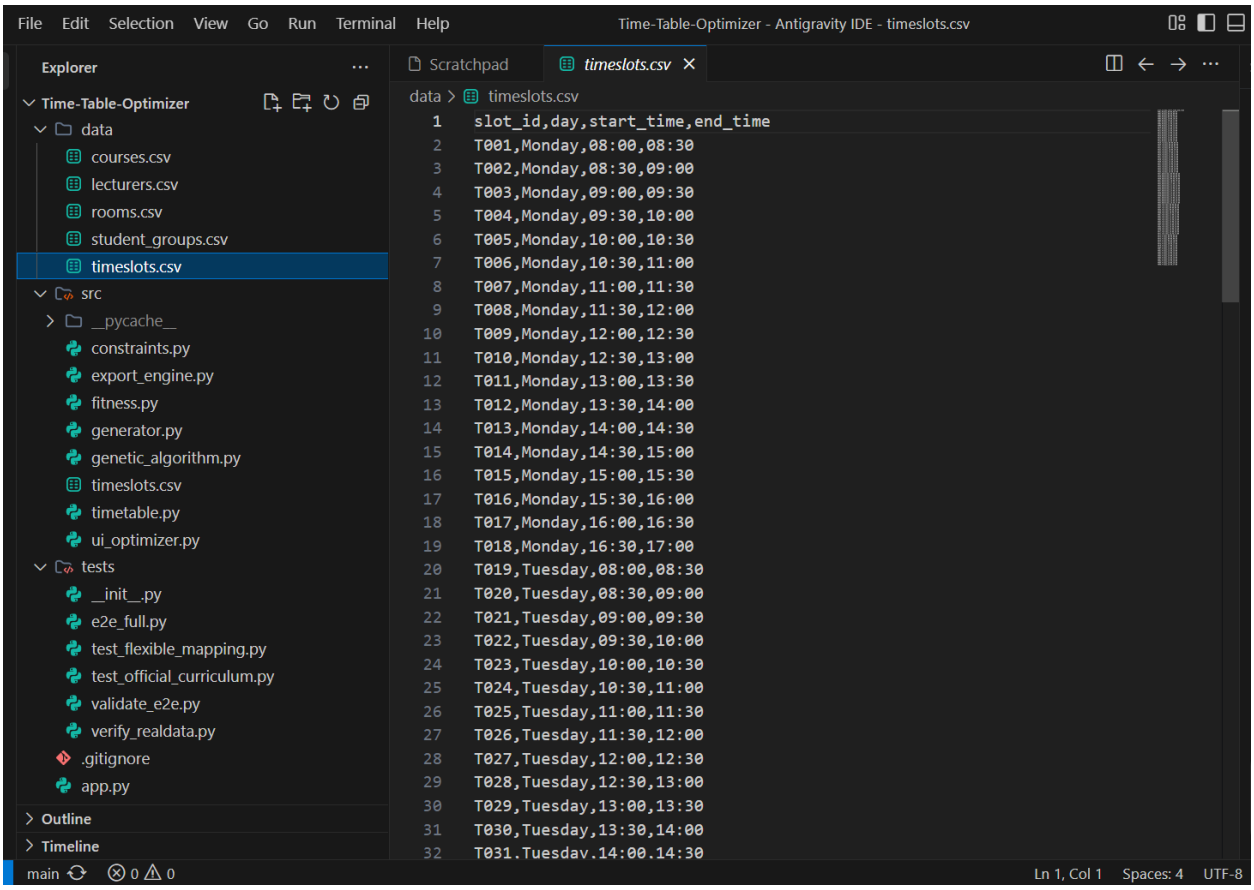
**Student Signature:** _ **Date:** 2026/10/17

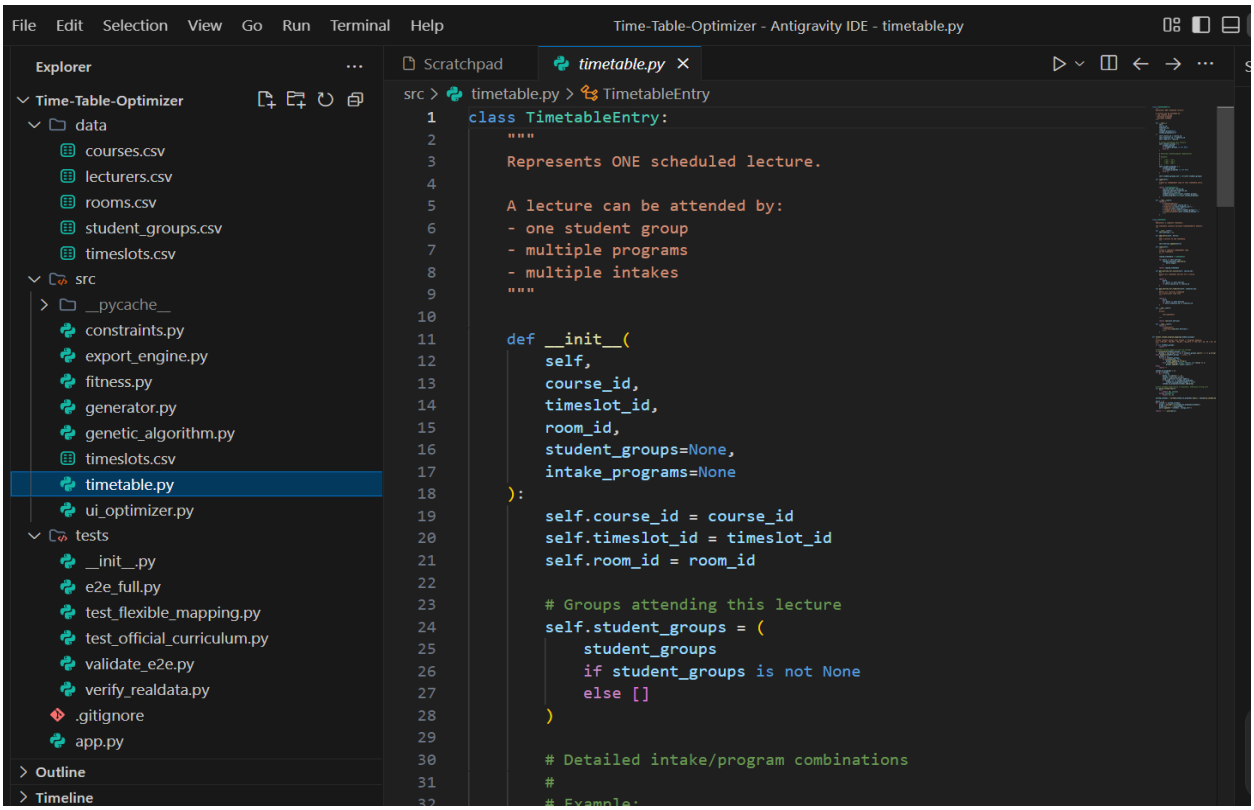
*Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.*

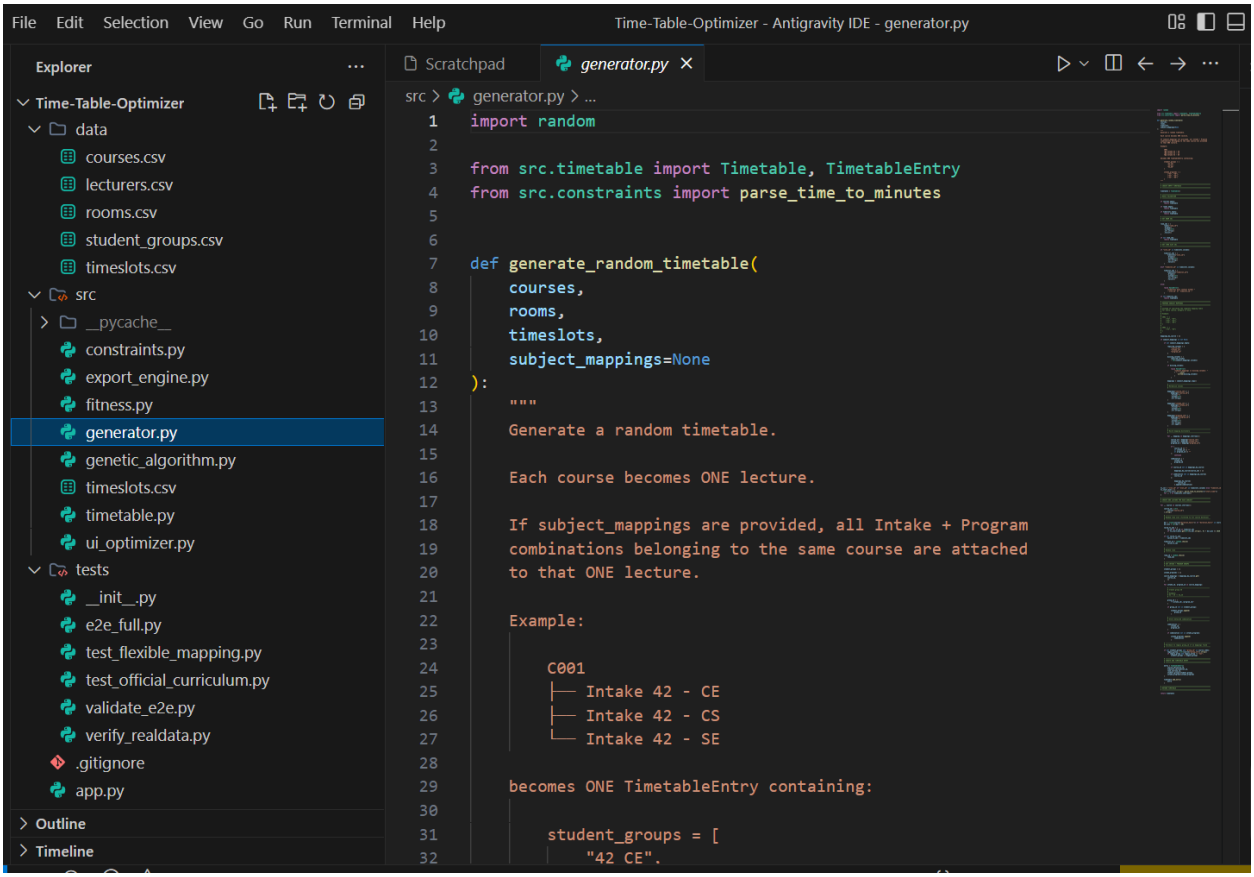
## 1. courses, lecturers, rooms, student groups and timeslots

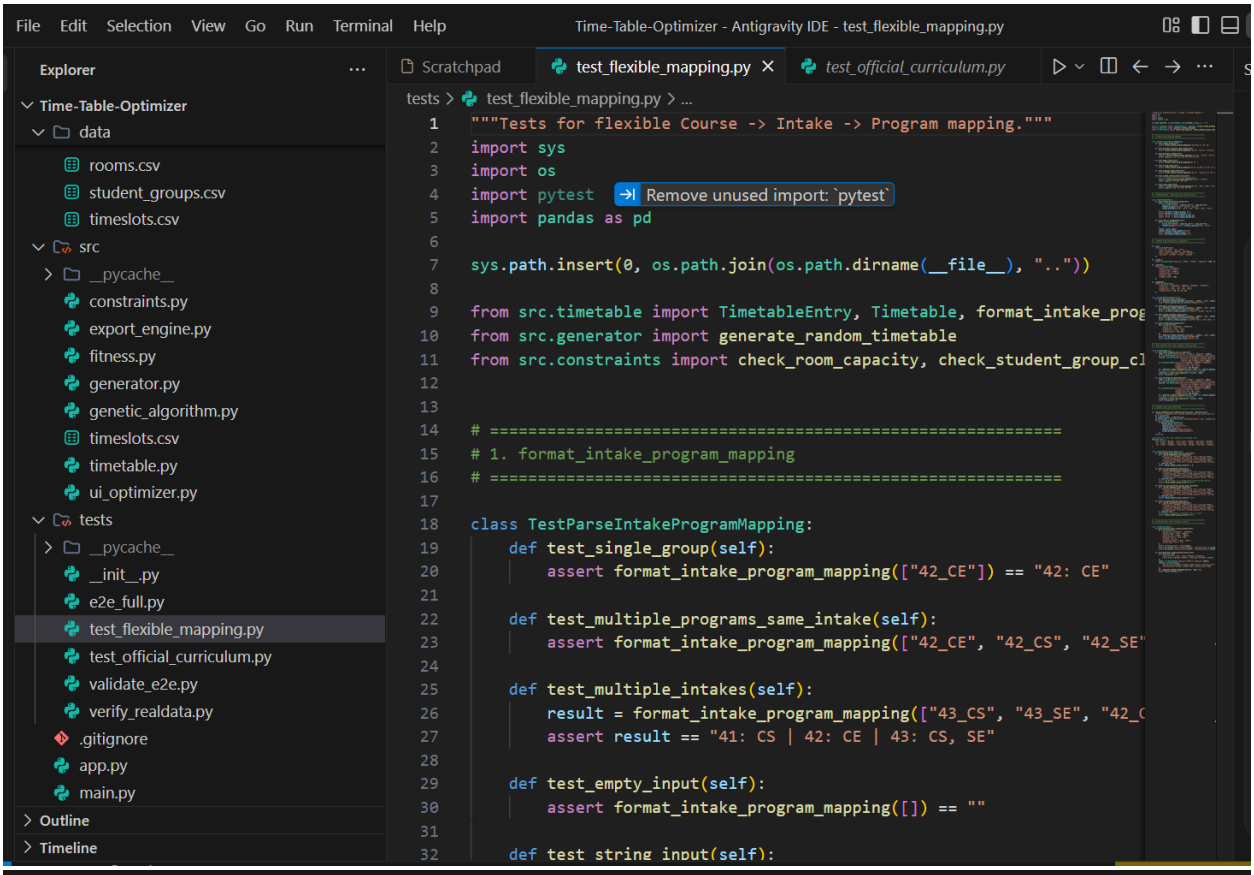


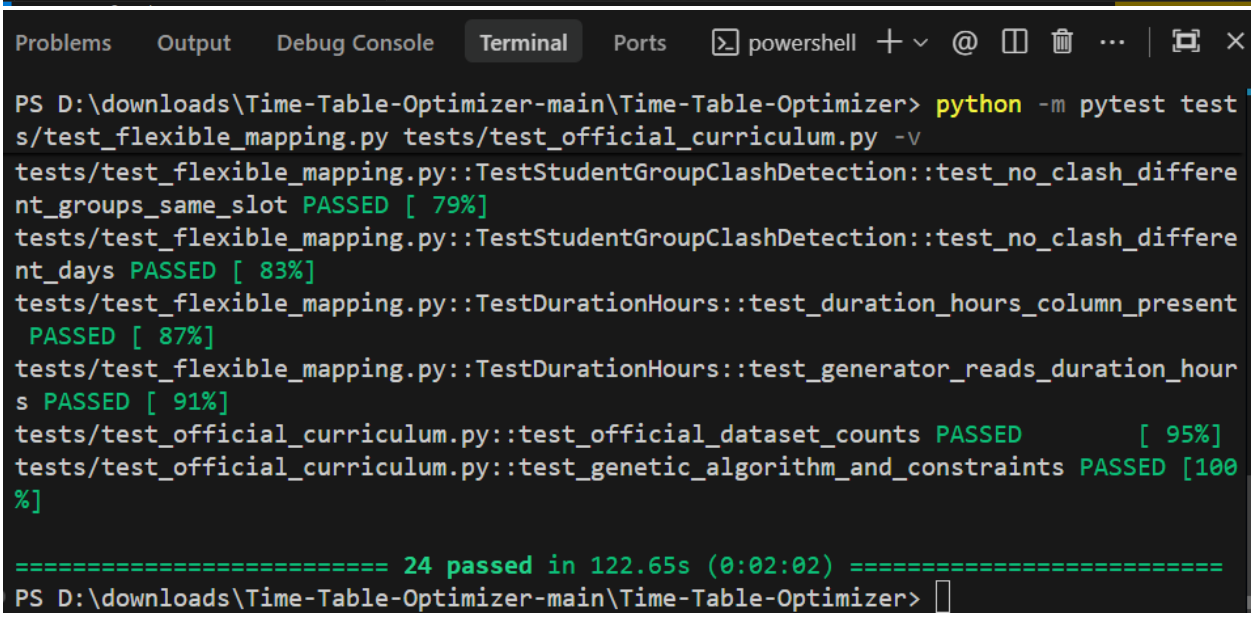




File Edit Selection View Go Run Terminal Help
Time-Table-Optimizer - Antigravity IDE - timeslots.csv
Explorer
Time-Table-Optimizer
data
courses.csv
lecturers.csv
rooms.csv
student_groups.csv
timeslots.csv
src
__pycache__
constraints.py
export_engine.py
fitness.py
generator.py
genetic_algorithm.py
timeslots.csv
timetable.py
ui_optimizer.py
tests
__init__.py
e2e_full.py
test_flexible_mapping.py
test_official_curriculum.py
validate_e2e.py
verify_realdata.py
.gitignore
app.py
Outline
Timeline
Scratchpad
timeslots.csv
data > timeslots.csv
slot_id,day,start_time,end_time
T001,Monday,08:00,08:30
T002,Monday,08:30,09:00
T003,Monday,09:00,09:30
T004,Monday,09:30,10:00
T005,Monday,10:00,10:30
T006,Monday,10:30,11:00
T007,Monday,11:00,11:30
T008,Monday,11:30,12:00
T009,Monday,12:00,12:30
T010,Monday,12:30,13:00
T011,Monday,13:00,13:30
T012,Monday,13:30,14:00
T013,Monday,14:00,14:30
T014,Monday,14:30,15:00
T015,Monday,15:00,15:30
T016,Monday,15:30,16:00
T017,Monday,16:00,16:30
T018,Monday,16:30,17:00
T019,Tuesday,08:00,08:30
T020,Tuesday,08:30,09:00
T021,Tuesday,09:00,09:30
T022,Tuesday,09:30,10:00
T023,Tuesday,10:00,10:30
T024,Tuesday,10:30,11:00
T025,Tuesday,11:00,11:30
T026,Tuesday,11:30,12:00
T027,Tuesday,12:00,12:30
T028,Tuesday,12:30,13:00
T029,Tuesday,13:00,13:30
T030,Tuesday,13:30,14:00
T031,Tuesday,14:00,14:30
main
0 0
Ln 1, Col 1
Spaces: 4
UTF-8


File Edit Selection View Go Run Terminal Help
Time-Table-Optimizer - Antigravity IDE - timetable.py
Explorer
Time-Table-Optimizer
data
courses.csv
lecturers.csv
rooms.csv
student_groups.csv
timeslots.csv
src
__pycache__
constraints.py
export_engine.py
fitness.py
generator.py
genetic_algorithm.py
timeslots.csv
timetable.py
ui_optimizer.py
tests
__init__.py
e2e_full.py
test_flexible_mapping.py
test_official_curriculum.py
validate_e2e.py
verify_realdata.py
.gitignore
app.py
Outline
Timeline
Scratchpad
timetable.py
src > timetable.py > TimetableEntry
class TimetableEntry:
"""
Represents ONE scheduled lecture.
A lecture can be attended by:
- one student group
- multiple programs
- multiple intakes
"""
def __init__(
self,
course_id,
timeslot_id,
room_id,
student_groups=None,
intake_programs=None
):
self.course_id = course_id
self.timeslot_id = timeslot_id
self.room_id = room_id
# Groups attending this lecture
self.student_groups = (
student_groups
if student_groups is not None
else []
)
# Detailed intake/program combinations
#


File Edit Selection View Go Run Terminal Help
Time-Table-Optimizer - Antigravity IDE - generator.py
Explorer
Time-Table-Optimizer
data
courses.csv
lecturers.csv
rooms.csv
student_groups.csv
timeslots.csv
src
__pycache__
constraints.py
export_engine.py
fitness.py
generator.py
genetic_algorithm.py
timeslots.csv
timetable.py
ui_optimizer.py
tests
__init__.py
e2e_full.py
test_flexible_mapping.py
test_official_curriculum.py
validate_e2e.py
verify_realdata.py
.gitignore
app.py
Outline
Timeline
Scratchpad
generator.py
src > generator.py > ...
import random
from src.timetable import Timetable, TimetableEntry
from src.constraints import parse_time_to_minutes
def generate_random_timetable(
courses,
rooms,
timeslots,
subject_mappings=None
):
"""
Generate a random timetable.
Each course becomes ONE lecture.
If subject_mappings are provided, all Intake + Program
combinations belonging to the same course are attached
to that ONE lecture.
Example:
C001
├── Intake 42 - CE
├── Intake 42 - CS
└── Intake 42 - SE
becomes ONE TimetableEntry containing:
student_groups = [
"42 CE".


File Edit Selection View Go Run Terminal Help
Time-Table-Optimizer - Antigravity IDE - test_flexible_mapping.py
Explorer
Time-Table-Optimizer
data
rooms.csv
student_groups.csv
timeslots.csv
src
__pycache__
constraints.py
export_engine.py
fitness.py
generator.py
genetic_algorithm.py
timeslots.csv
timetable.py
ui_optimizer.py
tests
__pycache__
__init__.py
e2e_full.py
test_flexible_mapping.py
test_official_curriculum.py
validate_e2e.py
verify_realdata.py
.gitignore
app.py
main.py
Outline
Timeline
Scratchpad
test_flexible_mapping.py
test_official_curriculum.py
tests > test_flexible_mapping.py > ...
"""Tests for flexible Course -> Intake -> Program mapping."""
import sys
import os
import pytest
Remove unused import: `pytest`
import pandas as pd
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.timetable import TimetableEntry, Timetable, format_intake_prog
from src.generator import generate_random_timetable
from src.constraints import check_room_capacity, check_student_group_cl
# ============================================================
# 1. format_intake_program_mapping
# ============================================================
class TestParseIntakeProgramMapping:
def test_single_group(self):
assert format_intake_program_mapping(["42_CE"]) == "42: CE"
def test_multiple_programs_same_intake(self):
assert format_intake_program_mapping(["42_CE", "42_CS", "42_SE"
def test_multiple_intakes(self):
result = format_intake_program_mapping(["43_CS", "43_SE", "42_C
assert result == "41: CS | 42: CE | 43: CS, SE"
def test_empty_input(self):
assert format_intake_program_mapping([]) == ""
def test_string_input(self):


Problems Output Debug Console Terminal Ports powershell
PS D:\downloads\Time-Table-Optimizer-main\Time-Table-Optimizer> python -m pytest tests/test_flexible_mapping.py tests/test_official_curriculum.py -v
tests/test_flexible_mapping.py::TestStudentGroupClashDetection::test_no_clash_different_groups_same_slot PASSED [ 79%]
tests/test_flexible_mapping.py::TestStudentGroupClashDetection::test_no_clash_different_days PASSED [ 83%]
tests/test_flexible_mapping.py::TestDurationHours::test_duration_hours_column_present PASSED [ 87%]
tests/test_flexible_mapping.py::TestDurationHours::test_generator_reads_duration_hours PASSED [ 91%]
tests/test_official_curriculum.py::test_official_dataset_counts PASSED [ 95%]
tests/test_official_curriculum.py::test_genetic_algorithm_and_constraints PASSED [100%]
========== 24 passed in 122.65s (0:02:02) ==========
PS D:\downloads\Time-Table-Optimizer-main\Time-Table-Optimizer>

**ESSENTIALS OF ARTIFICIAL INTELLIGENCE**

**INDIVIDUAL CONTRIBUTION SHEET**

| | | | |
|---|---|---|---|
| **Student Name** | N.P.G.V.K.Naotunna | **Registration No.** | D-BIT-24-0001 |
| **Degree / Intake** | Intake 41 - IT | **Group No.** | 56 |
| **Project Title** | AI Timetable Optimizer | | |

**Technical Contributions:**

| Task / Component | My Contribution | Tools / AI Technique Used | Evidence / Output |
|---|---|---|---|
| GA Class Design | Built the GeneticAlgorithm class. It pre-computes course, room-capacity, lecturer, and time-slot lookups to reduce repeated Pandas DataFrame queries during fitness evaluation. | Python, Pandas, custom Genetic Algorithm implementation | src/genetic_algorithm.py (GeneticAlgorithm class) |
| Population Handling | Implemented create_initial_population() and evaluate_population() to generate and score a configurable population of candidate timetables | Genetic Algorithm: population initialization and fitness evaluation | src/genetic_algorithm.py (create_initial_population(), evaluate_population()) |
| Selection | Implemented select_parents() using **truncation selection**, | Genetic Algorithm: | src/genetic_algorithm.py (select_parents()) |

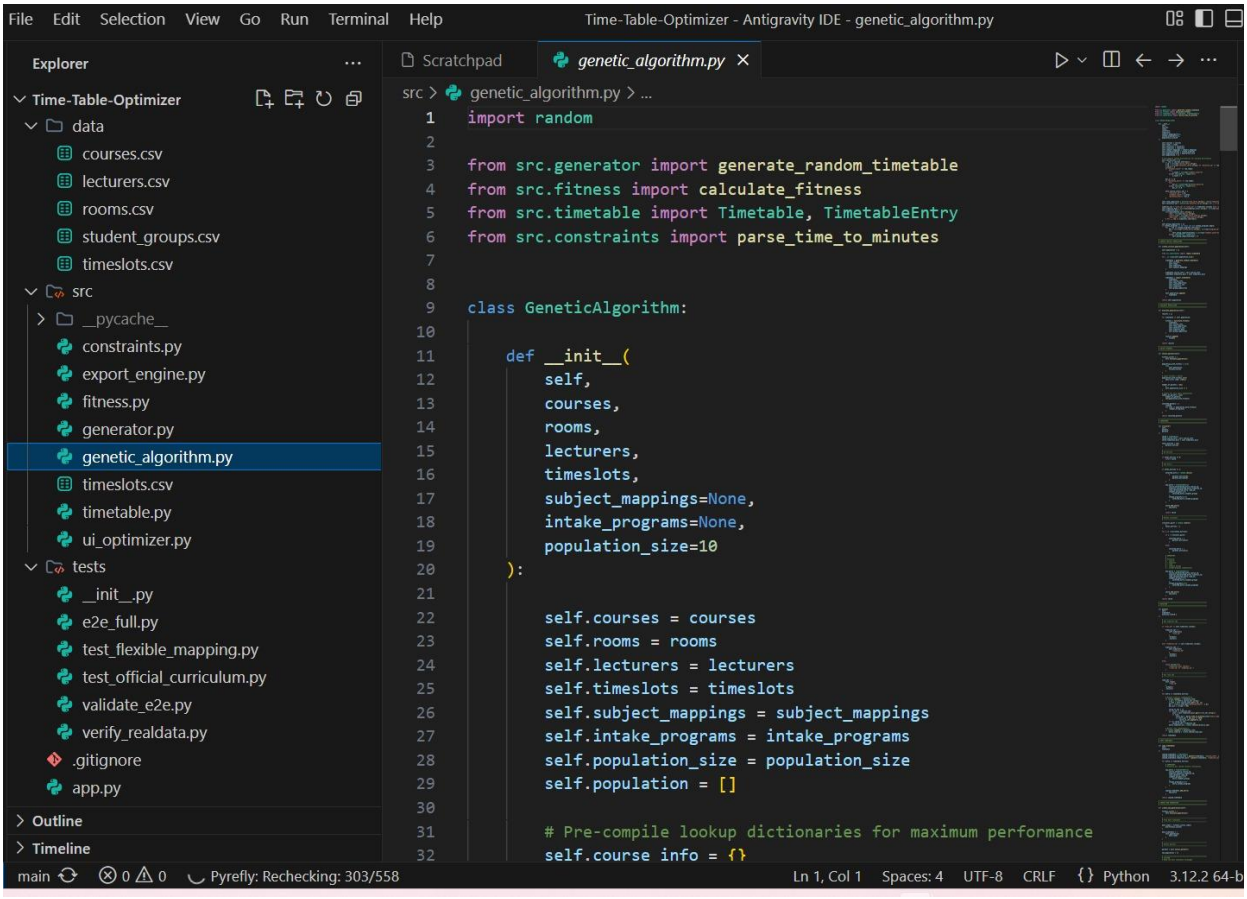
| | | | |
|---|---|---|---|
| | selecting the top 50% of the population with the lowest fitness values as the parent pool. | truncation selection | |
| Crossover and Mutation | Implemented single-point crossover with a crossover rate of 1.0 and point mutation with a mutation probability of 0.1, independently applied to timeslot and room attributes, to produce new timetables from selected parents. | Python, single-point crossover, point mutation | src/genetic_algorithm.py (crossover(), mutate()) |
| Generation Lifecycle, Elitism and Main Program | Implemented create_new_generation() with elitism to preserve the best individual from the current generation into the next. Developed main.py to load CSV data, run the generational loop, display fitness progress, and execute five constraint checks on the final best timetable. | Python, Pandas, elitism, generational optimization | src/genetic_algorithm.py (create_new_generation()), main.py (data loading, generation loop, and constraint checks) |

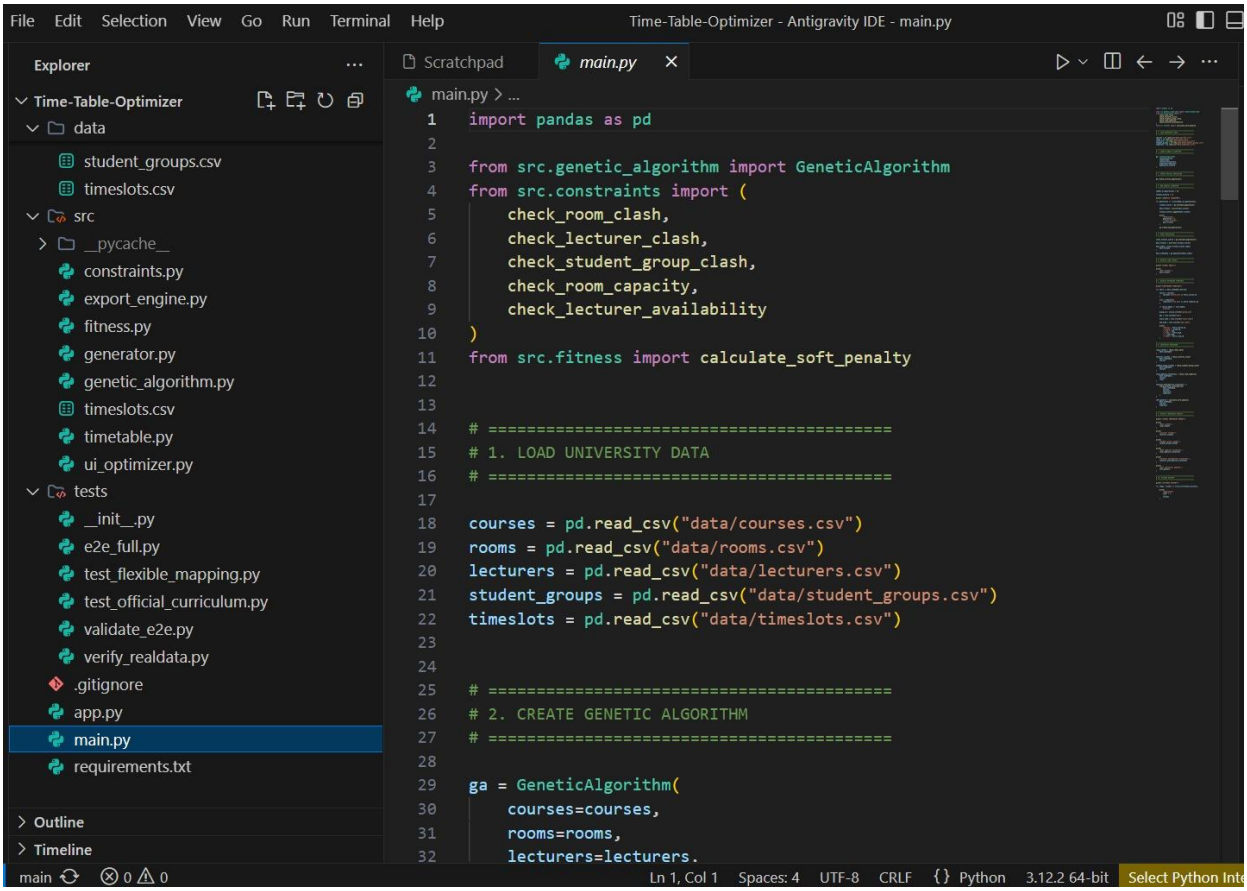
**Declaration**

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

**Student Signature:**  **Date:** 17-10-2026

*Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.*






## ESSENTIALS OF ARTIFICIAL INTELLIGENCE

## INDIVIDUAL CONTRIBUTION SHEET

| | | | |
|---|---|---|---|
| **Student Name** | T.M. Edirisinghe | **Registration No.** | D/COE/25/0021 |
| **Degree / Intake** | CE / 42 | **Group No.** | 56 |
| **Project Title** | AI Timetable Optimizer | | |

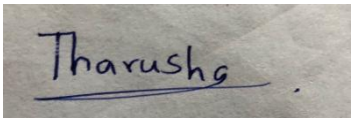
**Technical Contributions:**

| Task / Component | My Contribution | Tools / AI Technique Used | Evidence / Output |
|---|---|---|---|
| Interval-Overlap and Collision Detection | Implemented time-based interval-overlap calculations and clash | Python; interval-overlap calculations; constraint satisfaction | src/constraints.py: parse_time_to_minutes(), check_overlap(), |

| | | | |
|---|---|---|---|
| | detection for rooms, lecturers, and student groups, accounting for different lecture durations. | | check_room_clash(), check_lecturer_clash(), check_student_group_clash() |
| Resource Capacity and Constraint Validation | Implemented room-capacity validation by aggregating student cohort sizes, and checks to verify lecturer assignments against the registered lecturer roster. | Python; constraint checking; resource validation | src/constraints.py: check_room_capacity(), check_lecturer_availability() |
| Multi-Objective Fitness Calculation | Implemented fitness evaluation by combining hard-constraint violation penalties with soft-constraint penalties to assess timetable quality. Lower fitness indicates a better schedule. | Python; weighted fitness evaluation; optimization | src/fitness.py: calculate_fitness() |
| Soft Constraints and Timetable Quality | Implemented soft penalties for lunch-break overlaps, late finishing times, early starts, and high daily class counts to encourage more practical timetables. | Python; heuristic penalty modeling; soft constraints | src/fitness.py: calculate_soft_penalty() |
| Heuristic Local Conflict Repair | Implemented a local repair procedure that identifies timetable conflicts, reallocates capacity-violating room assignments, and searches alternative timeslots to reduce clashes. | Python; greedy local search; Min-Conflicts heuristic | src/constraints.py: repair_timetable(); integrated in src/genetic_algorithm.py |

**Declaration**

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

**Student Signature:**  **Date:** **10/17/2026**

*Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.*

## Interval-overlap and clash detection

C: > Users > Victus > Desktop > AI-Timetable-Optimizer > src > constraints.py

```python
import random

def parse_time_to_minutes(time_str):
    try:
        parts = time_str.split(":")
        return int(parts[0]) * 60 + int(parts[1])
    except Exception:
        return 0

def check_overlap(entry1, entry2, course_info, timeslots_dict):
    """
    Check if two timetable entries overlap in time on the same day.
    Uses: start_A < end_B AND start_B < end_A
    """
    slot1 = timeslots_dict.get(str(entry1.timeslot_id).strip())
    slot2 = timeslots_dict.get(str(entry2.timeslot_id).strip())
    if not slot1 or not slot2:
        return False

    if slot1["day"] != slot2["day"]:
        return False

    c1 = course_info.get(str(entry1.course_id).strip(), {})
    c2 = course_info.get(str(entry2.course_id).strip(), {})

    dur1 = float(c1.get("duration_hours", 1.0))
    dur2 = float(c2.get("duration_hours", 1.0))

    start1 = parse_time_to_minutes(slot1["start_time"])
    start2 = parse_time_to_minutes(slot2["start_time"])

    end1 = start1 + int(dur1 * 60)
    end2 = start2 + int(dur2 * 60)

    return start1 < end2 and start2 < end1
```

# 1) Room capacity and constraint validation

```python
def check_room_clash(timetable):
    load_fallback_data(timetable)
    clashes = 0
    entries = timetable.entries

    course_info = timetable.course_info
    timeslots_dict = timetable.timeslots_dict

    for i in range(len(entries)):
        for j in range(i + 1, len(entries)):
            entry1 = entries[i]
            entry2 = entries[j]

            if entry1.room_id == entry2.room_id:
                if check_overlap(entry1, entry2, course_info, timeslots_dict):
                    clashes += 1
    return clashes


def check_lecturer_clash(timetable, courses):
    load_fallback_data(timetable)
    clashes = 0
    entries = timetable.entries

    course_info = timetable.course_info
    timeslots_dict = timetable.timeslots_dict

    course_lecturers = {}
    if isinstance(courses, dict):
        sample = next(iter(courses.values())) if courses else None
        if isinstance(sample, dict):
            course_lecturers = {c_id: info.get("lecturer_id", "") for c_id, info in courses.items()}
        else:
            course_lecturers = courses
    elif hasattr(courses, "iterrows"):
        for _, row in courses.iterrows():
            course_lecturers[str(row["course_id"]).strip()] = str(row["lecturer_id"]).strip()
    else:
        # Fallback using our course_info dict
        course_lecturers = {c_id: info.get("lecturer_id", "") for c_id, info in course_info.items()}

    for i in range(len(entries)):
        for j in range(i + 1, len(entries)):
            entry1 = entries[i]
            entry2 = entries[j]

            lecturer1 = course_lecturers.get(entry1.course_id, "")
            lecturer2 = course_lecturers.get(entry2.course_id, "")

            if lecturer1 and lecturer2 and lecturer1 != "nan" and lecturer2 != "nan" and lecturer1 == lecturer2:
                if check_overlap(entry1, entry2, course_info, timeslots_dict):
                    clashes += 1
    return clashes
```

## 2) Fitness calculation

```python
def calculate_fitness(
    timetable,
    courses,
    rooms,
    lecturers,
    timeslots,
    intake_programs=None
):
    """
    Calculate the total fitness score.

    LOWER fitness = BETTER timetable.

    Fitness consists of:
        Hard constraint penalties
        +
        Soft constraint penalties
    """

    # Unpack courses once if it is the pre-compiled dict of dicts (course_info)
    if isinstance(courses, dict):
        sample = next(iter(courses.values())) if courses else None
        if isinstance(sample, dict):
            flat_course_lecturers = {c_id: info.get("lecturer_id", "") for c_id, info in courses.items()}
            flat_course_capacities = {c_id: info.get("student_count", 0) for c_id, info in courses.items()}
        else:
            flat_course_lecturers = courses
            flat_course_capacities = courses
    else:
        flat_course_lecturers = courses
        flat_course_capacities = courses

    # ======================================================
    # HARD CONSTRAINTS
    # ======================================================

    # 1. Room clashes
    room_clashes = check_room_clash(timetable)

    # 2. Lecturer clashes
    lecturer_clashes = check_lecturer_clash(timetable, flat_course_lecturers)

    # 3. Student group clashes
    student_group_clashes = check_student_group_clash(timetable, flat_course_lecturers)

    # 4. Room capacity
    room_capacity_violations = check_room_capacity(
        timetable,
        flat_course_capacities,
        rooms,
        intake_programs
    )

    # 5. Lecturer availability
    lecturer_availability_violations = (
        check_lecturer_availability(
            timetable,
            flat_course_lecturers,
            lecturers,
            timeslots
        )
    )
```

3) constraints

```python
def calculate_soft_penalty(
    timetable,
    courses,
    timeslots
):
    """
    Calculate penalties for soft constraints.

    Soft constraints do NOT make a timetable invalid.
    They simply make a timetable less desirable.
    """

    soft_penalty = 0

    # Load fallback info if missing
    load_fallback_data(timetable)
    course_info = timetable.course_info
    timeslots_dict = timetable.timeslots_dict

    # ====================================================
    # SOFT CONSTRAINT 1: Lunch Window Protection (12:00 - 12:30)
    # & Graduated Early-Finishing Preference
    #
    # Preferences:
    # 1. End time <= 14:30 (0 penalty) - Strongly preferred
    # 2. 14:30 < End time <= 15:30 (Small penalty: +10)
    # 3. 15:30 < End time <= 16:30 (Larger penalty: +35)
    # 4. End time > 16:30 (Very large penalty: +100)
    # 5. Lunch Break 12:00 - 12:30 protected (Overlap penalty: +100)
    # 6. Unnecessary 08:00 start tie-breaker (+2)
    # ====================================================

    LUNCH_START_MIN = 720  # 12:00
    LUNCH_END_MIN   = 750  # 12:30

    for entry in timetable.entries:
        slot = timeslots_dict.get(str(entry.timeslot_id).strip())
        if not slot:
            continue

        c_meta = course_info.get(str(entry.course_id).strip(), {})
        dur = float(c_meta.get("duration_hours", 1.0))

        start_min = parse_time_to_minutes(slot["start_time"])
        end_min = start_min + int(dur * 60)

        penalty = 0

        # 1. Lunch Protection: strongly discourage lectures overlapping 12:00 - 12:30
        if start_min < LUNCH_END_MIN and end_min > LUNCH_START_MIN:
            penalty += 250

        # 2. Early morning start (08:00) tie-breaker: discourage starting at 08:00
        if start_min < 510:  # 08:00
            penalty += 2

        # 3. Graduated Early-Finishing Preference (based on lecture END TIME)
        if end_min <= 870:      # Finishes by 14:30 (Ideal)
            pass
        elif end_min <= 930:    # Finishes by 15:30 (Acceptable for 3h or tight constraints)
            penalty += 12
        elif end_min <= 990:    # Finishes by 16:30 (Discouraged)
            penalty += 45
        else:                   # Finishes after 16:30 (Strongly discouraged)
            penalty += 120

        soft_penalty += penalty
```

4) Local conflict repair



```
def repair_timetable(timetable, courses, rooms, lecturers, timeslots, intake_programs=None):
    load_fallback_data(timetable)
    if hasattr(rooms, "iterrows"):
        room_details = []
        for _, r in rooms.iterrows():
            room_details.append({
                "room_id": str(r["room_id"]).strip(),
                "capacity": int(r["capacity"])
            })
    elif isinstance(rooms, dict):
        room_details = [{"room_id": k, "capacity": v} for k, v in rooms.items()]
    else:
        room_details = rooms

    if hasattr(timeslots, "iterrows"):
        if "slot_id" in timeslots.columns:
            timeslot_ids = timeslots["slot_id"].dropna().astype(str).str.strip().tolist()
        else:
            timeslot_ids = timeslots["timeslot_id"].dropna().astype(str).str.strip().tolist()
    else:
        timeslot_ids = timeslots

    course_info = timetable.course_info
    timeslots_dict = timetable.timeslots_dict

    course_lecturers = {}
    course_capacities = {}
    if isinstance(courses, dict):
        sample = next(iter(courses.values())) if courses else None
        if isinstance(sample, dict):
            course_lecturers = {c_id: info.get("lecturer_id", "") for c_id, info in courses.items()}
            course_capacities = {c_id: info.get("student_count", 0) for c_id, info in courses.items()}
        else:
            course_lecturers = courses
            course_capacities = {}
    elif hasattr(courses, "iterrows"):
        course_lecturers = {str(row["course_id"]).strip(): str(row["lecturer_id"]).strip() for _, row in courses.iterrows()}
        for _, row in courses.iterrows():
            c_id = str(row["course_id"]).strip()
            s_count = 0
            if "student_count" in row.index:
                try:
                    s_count = int(row["student_count"])
                except (ValueError, TypeError):
                    s_count = 0
            course_capacities[c_id] = s_count
    else:
        course_lecturers = {c_id: info.get("lecturer_id", "") for c_id, info in course_info.items()}
        course_capacities = {c_id: info.get("student_count", 0) for c_id, info in course_info.items()}

    group_capacities = {}
    if intake_programs is not None:
        if hasattr(intake_programs, "iterrows"):
            for _, row in intake_programs.iterrows():
                key = (str(row["intake_id"]).strip(), str(row["program_id"]).strip().upper())
                try:
                    group_capacities[key] = int(row["student_count"])
                except (ValueError, TypeError):
                    group_capacities[key] = 0
        elif isinstance(intake_programs, dict):
            group_capacities = intake_programs

    def get_clashes_for_entry(ent, idx):
        ent_groups = set(ent.student_groups) if ent.student_groups else set()
        ent_lecturer = course_lecturers.get(ent.course_id, "")
```

**ESSENTIALS OF ARTIFICIAL INTELLIGENCE**

**INDIVIDUAL CONTRIBUTION SHEET**

| | | | |
|---|---|---|---|
| **Student Name** | JNL Asirini | **Registration No.** | D/BSE/25/0024 |
| **Degree / Intake** | SE- Intake 42 | **Group No.** | 56 |
| **Project Title** | Intelligent Multi-Intake University Timetable Optimization System Using Genetic Algorithms | | |

**Technical Contributions:**

| Task / Component | My Contribution | Tools / AI Technique Used | Evidence / Output |
|---|---|---|---|
| Streamlit UI (app.py) | Built the interface with GA settings, live progress, timetable views and filters. | Python, Streamlit, pandas, CSS | app.py |
| GA control (ui_optimizer.py) | Connected the UI to the GA; progress_callback shows live generation and fitness. | Genetic Algorithm, st.progress | ui_optimizer.py; 0 clashes |
| Validation dashboard | Displays the constraint audit results (room, lecturer, student-group, capacity) from the GA. | Streamlit, st.metric, custom CSS | Validation page |
| Export engine (export_engine.py) | Exports timetables to CSV, PDF and Excel, fixed merged-cell error. | ReportLab, OpenPyXL | Exported PDF and XLSX files |

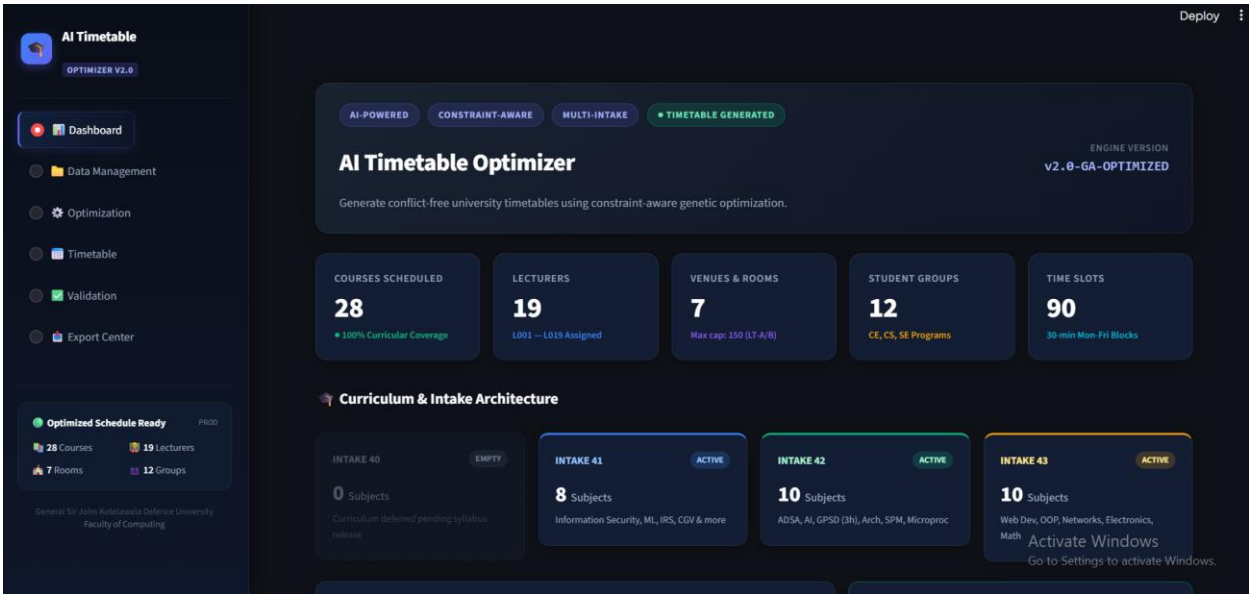
**Declaration**

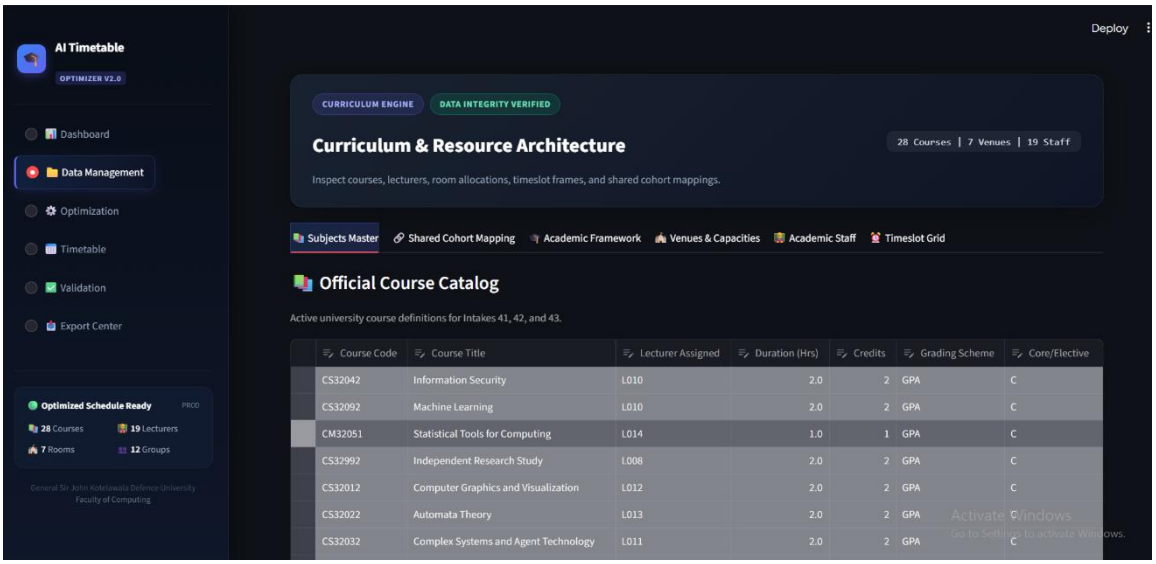
I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

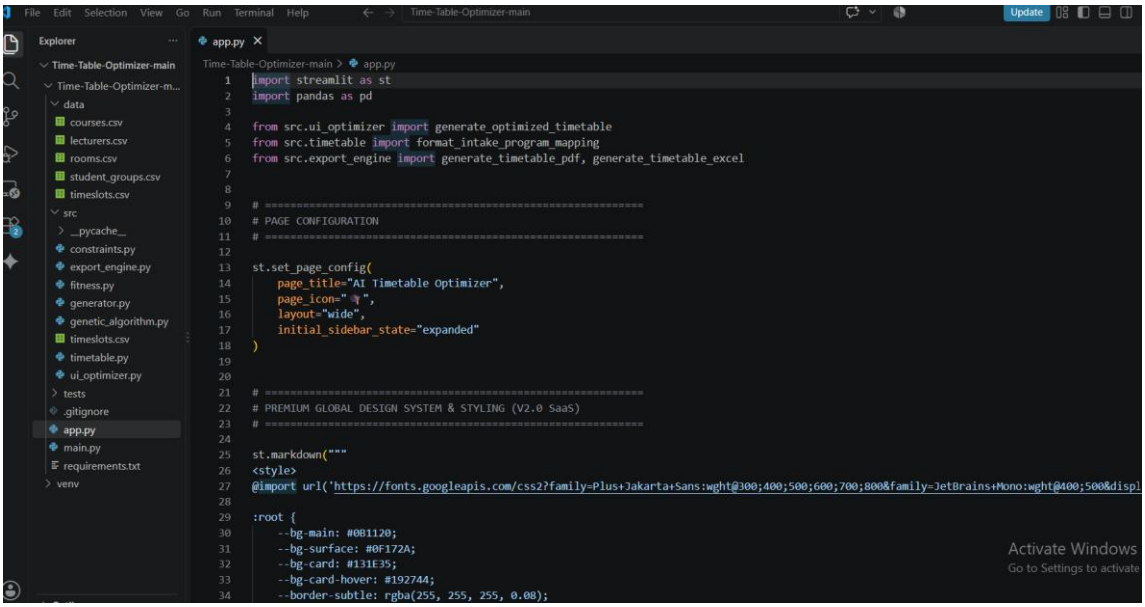
**Student Signature:**  **Date:** 2026/10/17

*Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.*

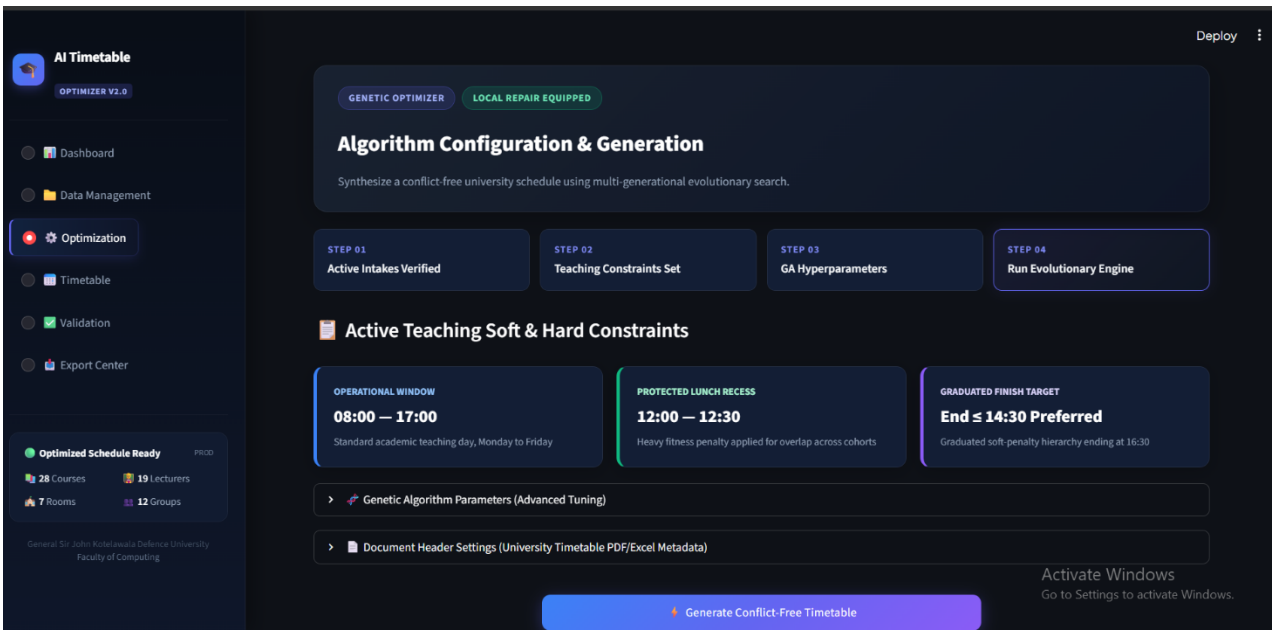
## Streamlit UI

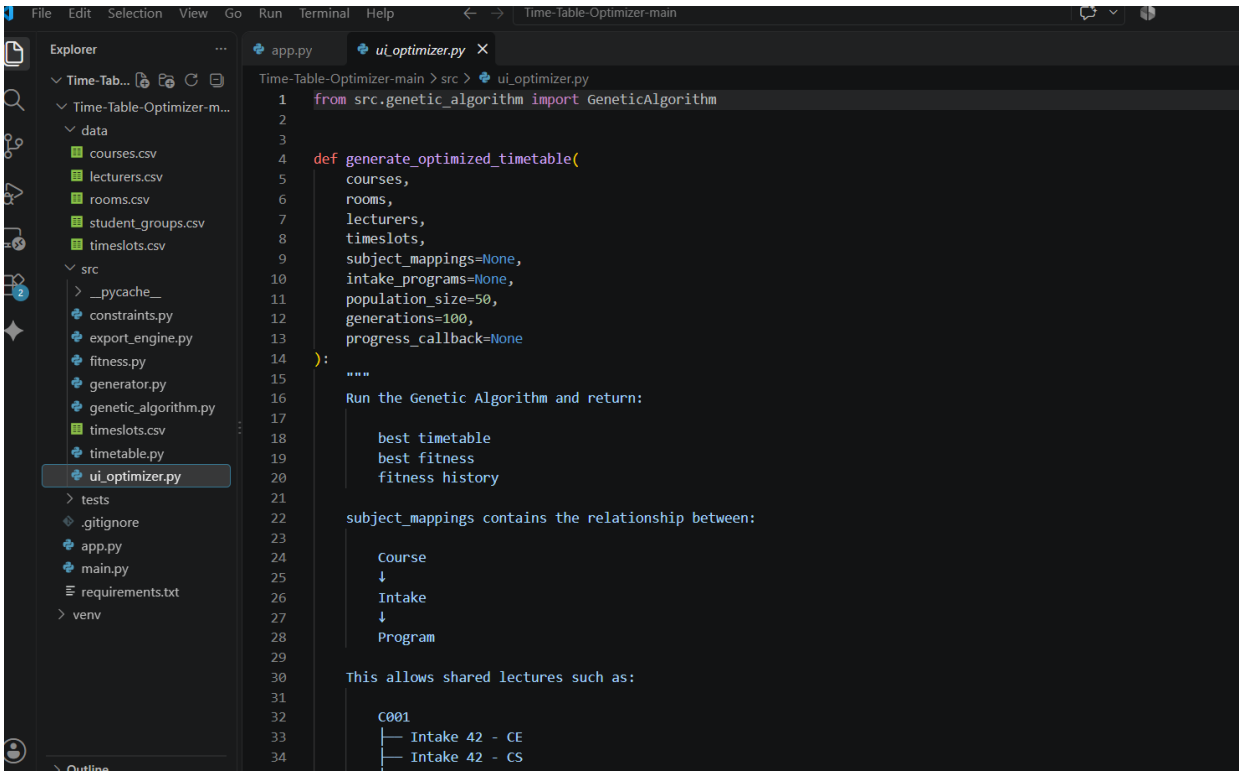


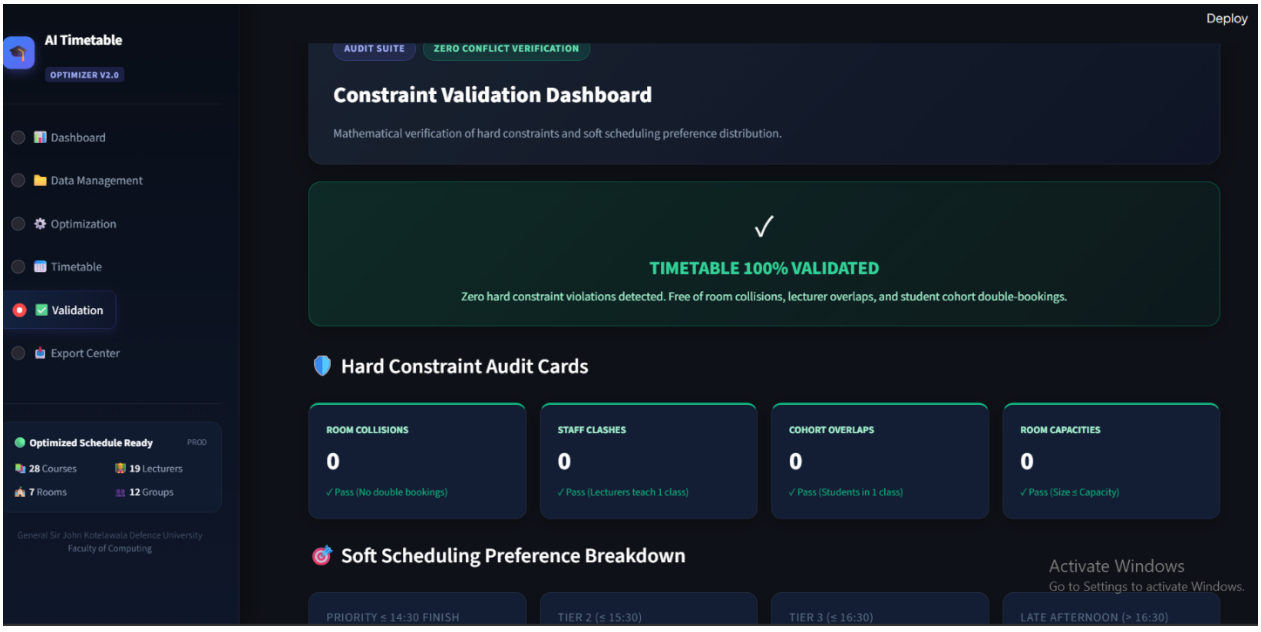

File Edit Selection View Go Run Terminal Help
Time-Table-Optimizer-main
Update
Explorer
app.py
Time-Table-Optimizer-main > app.py
Time-Table-Optimizer-main
Time-Table-Optimizer-m...
data
courses.csv
lecturers.csv
rooms.csv
student_groups.csv
timeslots.csv
src
__pycache__
constraints.py
export_engine.py
fitness.py
generator.py
genetic_algorithm.py
timeslots.csv
timetable.py
ui_optimizer.py
tests
.gitignore
app.py
main.py
requirements.txt
venv
import streamlit as st
import pandas as pd
from src.ui_optimizer import generate_optimized_timetable
from src.timetable import format_intake_program_mapping
from src.export_engine import generate_timetable_pdf, generate_timetable_excel
# PAGE CONFIGURATION
st.set_page_config(
page_title="AI Timetable Optimizer",
page_icon="
layout="wide",
initial_sidebar_state="expanded"
)
# PREMIUM GLOBAL DESIGN SYSTEM & STYLING (V2.0 SaaS)
st.markdown("""
<style>
@import url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700;800&family=JetBrains+Mono:wght@400;500&displ
:root {
--bg-main: #0B1120;
--bg-surface: #0F172A;
--bg-card: #131E35;
--bg-card-hover: #192744;
--border-subtle: rgba(255, 255, 255, 0.08);
Activate Windows
Go to Settings to activate

## GA control


AI Timetable
OPTIMIZER V2.0
Deploy
Dashboard
Data Management
Optimization
Timetable
Validation
Export Center
GENETIC OPTIMIZER
LOCAL REPAIR EQUIPPED
Algorithm Configuration & Generation
Synthesize a conflict-free university schedule using multi-generational evolutionary search.
STEP 01
Active Intakes Verified
STEP 02
Teaching Constraints Set
STEP 03
GA Hyperparameters
STEP 04
Run Evolutionary Engine
Active Teaching Soft & Hard Constraints
OPERATIONAL WINDOW
08:00 — 17:00
Standard academic teaching day, Monday to Friday
PROTECTED LUNCH RECESS
12:00 — 12:30
Heavy fitness penalty applied for overlap across cohorts
GRADUATED FINISH TARGET
End ≤ 14:30 Preferred
Graduated soft-penalty hierarchy ending at 16:30
Genetic Algorithm Parameters (Advanced Tuning)
Document Header Settings (University Timetable PDF/Excel Metadata)
Optimized Schedule Ready
PROD
28 Courses
19 Lecturers
7 Rooms
12 Groups
General Sir John Kotelawala Defence University
Faculty of Computing
Generate Conflict-Free Timetable
Activate Windows
Go to Settings to activate Windows.




## Validation dashboard



## Export engine

